COMP302

GROUP 6 PROJECT

“DUNGEON KURAWLER”

REPORT

Ali Güven Gençer

Bilge Sena Gündebahar

Bora Toker

Hilal Aygül

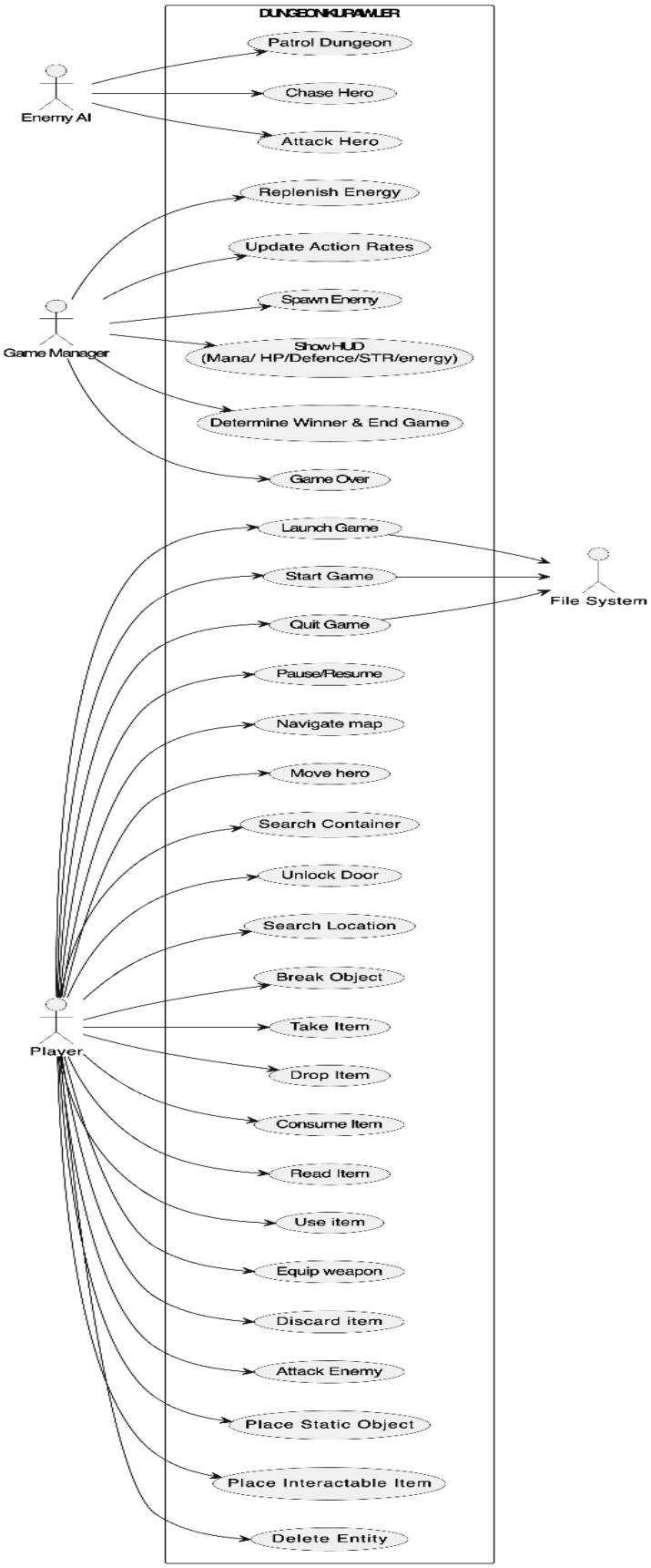
Mert Kalender

Yusuf Emir Doğan

KOÇ
ÜNİVERSİTESİ
COLLEGE OF ENGINEERING

Table of Contents:

UML Use Case Diagram:



Dungeon KUrawler - Use Case Narratives:

🖥️ System Initialization & Design Mode

UC-01 - Launch Game

- **Primary Actor:** Player
- **Secondary Actor:** OS (Operating System) / File System
- **Description:** Covers the initial execution of the application from the moment the user runs the program until the Main Menu is fully initialized and rendered.
- **Pre-conditions:** The game executable is run on a system with a valid Java Runtime Environment (JRE).
- **Success Guarantees (Post-conditions):**
    - The application window opens successfully using Java Swing `JFrame` managed by a `CardLayout`.
    - The Main Menu (`MainMenuView`) is displayed with active buttons: **Start Game**, **Load Game**, **Help**, and **Quit Game**.
- **Normal Flow:**
    - The Player initiates the application execution.
    - The System requests window framework resources from the operating system.
    - The System loads all core global assets (UI textures, fonts, basic character sprites) from the File System.
    - The System initializes the parent frame and builds the default `MainMenuView` GUI components.
    - The System renders the Main Menu layout on the display screen.
    - The System enters an idle event-listening state, awaiting Player interaction.
- **Extensions (Alternative Flows):**
    - **3a. Missing Core Assets:**
        1. The System fails to locate critical UI textures or fallback fonts.
        2. The System displays a fatal error alert dialog box and cleanly terminates the process.

UC-02 - Start Game

- **Primary Actor:** Player / Level Designer
- **Secondary Actor:** File System
- **Description:** Describes how a player initializes a new session, starting within **Design Mode** to shape or auto-generate the map before seamlessly launching into **Play Mode**.
- **Pre-conditions:** The application is running, and the active view is the Main Menu.
- **Success Guarantees (Post-conditions):**
    - A valid 28x20 dungeon grid layer is fully structured.
    - The system seamlessly switches states into **Play Mode** (Adventure Mode).
    - Hero entity is successfully spawned with persistent starting metrics.
- **Normal Flow:**
    - The Player clicks the **Start Game** button on the Main Menu interface.
    - The System transitions the active window state to **Design Mode** (`DesignModeView`).

    - The System displays the asset selection panel ("item carpet") loaded with walls, columns, generic enemies, potions, and containers.
    - The Designer either manually drags and drops assets onto the 28x20 grid or clicks **Random Generate Map** for automated procedural layout generation.
    - The Designer completes the layout and clicks the **Play** button at the bottom navigation panel.
    - The System runs map-validation rules, fixes the static Level Door at boundary coordinates `(11, 0)`, updates the surrounding wall tiles, and establishes the initial Level Key placement parameters.
    - The System swaps active states to **Play Mode**, instantiates the Hero, and boots the runtime loop.
- **Extensions (Alternative Flows):**
    - **6a. Invalid Map Specifications:**
        1. The map layout validation fails (e.g., missing critical coordinates or isolated nodes).
        2. The System intercepts the progression and prompts the user with an error notification popup.

⚔️ Hero Progression & Play Mode Dynamics

UC-03 - Explore Dungeon (Move & Navigate)

- **Primary Actor:** Player (Hero)
- **Secondary Actor:** Game Clock
- **Description:** Outlines the core movement mechanics across the 28x20 grid tiles, including collision evaluation and action resource budgeting.
- **Pre-conditions:** The application is in **Play Mode**, the Hero is active, and current Energy is ≥ 3.
- **Success Guarantees (Post-conditions):**
    - The Hero's 2D coordinate values are modified on the grid matrix.
    - Action energy reserves are decremented accordingly.
- **Normal Flow:**
    - The Player presses a registered movement input key (`WASD` or Arrow keys).
    - The System evaluates the target grid cell properties to confirm it is passable (e.g., an empty floor tile, an open/unlocked doorway).
    - The System moves the Hero entity to the validated destination coordinates.
    - The System deducts exactly **3 units of Energy** from the Hero's active balance.
    - The System checks if the new coordinate position matches the unlocked Level Door `(11, 0)` to check for level transition triggers.
    - The System updates the HUD elements and flags the Game Clock to advance a movement frame tick.
- **Extensions (Alternative Flows):**
    - **2a. Impassable Object Collision:**
        1. The targeted cell contains a rigid obstacle (Wall, Column, or a locked Door structure).

2. The System nullifies the movement request; no position updates occur and no Energy is deducted.
    - **4a. Starved Energy State:**
        1. The Hero's current Energy balance drops below the required 3 units.
        2. The System locks out movement commands until the periodic recovery loop processes.

UC-04 - Interact with Object

- **Primary Actor:** Player (Hero)
- **Secondary Actor:** None
- **Description:** Handles spatial interaction workflows between the Hero and neighboring container units or interactive tiles.
- **Pre-conditions:** The application is in **Play Mode**, and the Hero is standing immediately adjacent to an interactive object.
- **Success Guarantees (Post-conditions):**
    - The targeted object's internal state is modified (opened, unlocked, or emptied).
    - Popups, dialog vectors, or level transitions are initialized based on item selection.
- **Normal Flow:**
    - The Player positions the Hero next to an interactive object (Chest, Crate, Door) and triggers the interaction key (`E`).
    - The System detects the entity type and renders a context-aware Java Swing dialogue panel overlay.
    - The Player selects their desired choice option (e.g., "Open" a chest, "Search" a crate, or "Enter New Level" through an open door).
    - The System validates the inventory conditions (e.g., verifying a standard `KeyItem` for chests, or the level-specific `LevelKey` for Level Doors).
    - The System updates the state of the target entity, dispenses contents to the floor/inventory, or launches a level generation phase.
    - The System refreshes the graphic canvas rendering.
- **Extensions (Alternative Flows):**
    - **4a. Missing Required Key Constraints:**
        1. The Hero initiates an interaction on a locked object without possessing the required key type in their active hotbar slots.
        2. The System cancels the execution and prints a diagnostic warning: *"Locked. Corresponding key required."*

UC-05 - Attack Enemy

- **Primary Actor:** Player (Hero)
- **Secondary Actor:** Enemy AI
- **Description:** Manages tactical offensive combat interactions directed at entities occupying immediate neighboring tiles.
- **Pre-conditions:** Play Mode is active, an enemy target is adjacent, and the attack action cooldown state is clear.
- **Success Guarantees (Post-conditions):**

    - The targeted enemy's Hit Points (HP) are reduced.
    - Killed enemies are cleared from memory matrices and leave loot remains.
- **Normal Flow:**
    - The Player hovers/targets an adjacent enemy entity and executes an attack command.
    - The System computes damage calculations utilizing the formula:
    - $$\text{Damage} = \left(\frac{\text{STR}}{2}\right) + \text{WeaponAtk} - \text{EnemyDefense}$$
    - The System subtracts the resultant damage from the enemy's current HP pool and triggers floating combat visual cues.
    - If the enemy survives the impact, its state is updated to be eligible for retaliation behaviors on the next tick cycle.
    - If the enemy's HP falls ≤ 0, the System purges the entity from the grid layer and rolls item drop drop-tables onto that tile coordinate.
- **Extensions (Alternative Flows):**
    - **5a. Final Boss Entity Annihilation:**
        1. The 2x2 Final Boss entity's HP drops to absolute zero.
        2. The System suppresses standard random drop tables and spawns the mission-critical `VictoryCoin` on its center coordinate.

UC-06 - Manage Inventory

- **Primary Actor:** Player (Hero)
- **Description:** Outlines slot management, active usage, and gear equipment processing within the Hero's 8-slot action hotbar array.
- **Pre-conditions:** The application runtime is actively handling **Play Mode**.
- **Success Guarantees (Post-conditions):**
    1. Items are consumed, shifted, equipped, or dropped onto map floor tiles correctly.
    2. Character core attributes (HP, Mana, STR, DEF) are instantly modulated according to equipment stat profiles.
- **Normal Flow:**
    1. The Player scrolls the mouse wheel or keys a hotkey (`1-8`) to select a specific slot index.
    2. To consume an item (such as a health/mana potion or a speed scroll), the Player activates the selected item.
    3. The System processes the item's attribute payloads (e.g., healing HP pools, modifying mana scales) and decrements the item stack count.
    4. To equip gear pieces, the Player interacts with a piece of wearable equipment (Weapon, Armor, or Ring).
    5. The System transfers the item to the dedicated equipment slot, shifting modifiers, and recalculating stat sheets.
    6. To discard, the Player executes a drop instruction; the object is instant-spawned onto the floor tile currently occupied by the Hero.

🤖 Automations & Ambient Game Loops

UC-07 - Quit Game

- **Primary Actor:** Player
- **Secondary Actor:** OS / File System
- **Description:** Allows the user to gracefully suspend the active execution state and return safely to the system workspace or parent menus.
- **Pre-conditions:** The application process is running.
- **Success Guarantees (Post-conditions):**
    1. Frame loops are decoupled, graphics contexts are cleared, and file access points are closed.
- **Normal Flow:**
    1. The Player fires a exit escape command or clicks **Quit Game** from the Main Menu panel.
    2. The System triggers a modal prompt window requesting a validation signature.
    3. The Player clicks confirm.
    4. The System stops background timing engines, de-allocates display structures, and completes process termination.

UC-08 - Create Custom Map

- **Primary Actor:** Level Designer
- **Description:** Governs design matrix workflows, allowing the modification of grid tile properties and structural object coordinates inside Design Mode.
- **Pre-conditions:** The application state is loaded inside the **DesignModeView**.
- **Success Guarantees (Post-conditions):**
    - Target entity descriptors are bound onto the selected 28x20 array coordinates.
- **Normal Flow:**
    - The Designer clicks an item component from the item selection deck.
    - The Designer left-clicks an empty target grid tile on the layout grid.
    - The System checks spatial validity rules and populates the tile with the selected object instance.
    - To undo or remove, the Designer right-clicks an occupied cell to wipe its structure back to default floor properties.
- **Extensions (Alternative Flows):**
    - **3a. Direct Nesting to Containers:**
        1. The Designer drops an item entity directly over a pre-existing Chest or Crate node.
        2. The System catches the collision and pushes that item into the target container's internal inventory array.

UC-09 - Patrol Dungeon

- **Primary Actor:** Enemy AI
- **Secondary Actor:** Game Clock
- **Description:** Dictates default pathing behaviors for active hostile entities moving across the map layout when out of combat engagement range.

- **Pre-conditions:** An enemy actor instance is alive and loaded within the map coordinates.
- **Success Guarantees (Post-conditions):**
    - Target coordinates are updated based on patrol constraint rules.
- **Normal Flow:**
    - The Game Clock fires an automated action execution tick.
    - The System checks spatial proximity constraints relative to the Hero's location.
    - If the tile distance calculation returns > 5 units and the entity is a Knight, it calculates a valid random direction vector.
    - The System verifies the target cell properties and updates the Knight's coordinate location.
- **Extensions (Alternative Flows):**
    - **3a. Sorcerer Teleport Loop:**
        1. If the patrolling entity belongs to the Sorcerer sub-class, standard step calculations are bypassed. Instead, every 7 seconds, it runs a probability check to execute a teleport maneuver to an empty, passable tile on the map.

UC-10 - Chase Hero

- **Primary Actor:** Enemy AI
- **Secondary Actor:** Game Clock
- **Description:** Defines how walking enemy types path toward the Hero once aggression triggers are met.
- **Pre-conditions:** The Hero's current coordinates fall within an aggressive radius (≤ 5 tiles) of a pathing enemy type (Knight).
- **Success Guarantees (Post-conditions):**
    1. The enemy entity steps 1 tile unit closer to the Hero along an optimal path vector.
- **Normal Flow:**
    1. The Game Clock fires an aggressive unit action tick.
    2. The Knight entity validates that the Hero is within its ≤ 5 tile aggro bubble.
    3. The entity calculates an optimized directional step designed to minimize the Euclidean distance to the Hero's position.
    4. The System ensures the path step is clear of hard wall collisions and commits the movement step.

UC-11 - Attack Hero

- **Primary Actor:** Enemy AI
- **Secondary Actor:** Game Clock
- **Description:** Governs damage calculation workflows when an active enemy entity hits the Hero.
- **Pre-conditions:** The Hero entity occupies a tile coordinate that intersects with the active enemy's combat range framework.
- **Success Guarantees (Post-conditions):**
    - The Hero's health pool is decremented, and the UI status panel updates values instantly.
- **Normal Flow:**

    - The Enemy AI triggers an offensive combat turn.
    - The System confirms range constraints: adjacent cells for standard Knights, or unobstructed clear linear line-of-sight for Sorcerer spell trajectory logic.
    - The System evaluates damage formulas (base value of 4 for standard Knights; base value of 8 for Sorcerer projectiles) factored against the Hero's active DEF value.
    - The resultant damage decreases the Hero's current HP pool, triggering a HUD red-flash update.
- **Extensions (Alternative Flows):**
    - **2a. Final Boss Multi-Phase Assault:**
        1. The 2x2 Final Boss entity (Hakan Ayral) executes specialized high-tier attack loops and checks its HP thresholds to spawn protective minion clusters across surrounding tiles.

UC-12 - Replenish Energy

- **Primary Actor:** Game Clock
- **Description:** Handles automatic background replenishment of the Hero's core action energy resource over time.
- **Pre-conditions:** The application is running in **Play Mode**, the Hero is alive, and current energy $<$ maximum cap.
- **Success Guarantees (Post-conditions):**
    1. Hero's energy pool is safely incremented up to its maximum ceiling.
- **Normal Flow:**
    1. The Game Clock triggers an automated energy replenishment interval pulse.
    2. The System confirms the Hero's life state is active and verifies current energy is below maximum parameters.
    3. The System evaluates base recovery metrics (factoring in any equipped passive Ring item attributes) and adds energy points up to the hard ceiling cap.
    4. The System updates the active energy bar on the HUD layer.

UC-13 - Update Action Rates

- **Primary Actor:** Game Clock
- **Description:** Advances action resource gauges to award functional movement/combat turn tokens to registered entities.
- **Pre-conditions:** Play Mode is running.
- **Success Guarantees (Post-conditions):**
    1. Characters receive structural action authorization tokens as their internal metric loops hit completion metrics.
- **Normal Flow:**
    1. The Game Clock increments a computational unit cycle.
    2. The System cycles through all active entities present in the active scene data, checking individual velocity metrics ($\text{base speed} \pm \text{modifiers}$).
    3. The System appends these speed calculations directly to each entity's internal action accumulator gauge.
    4. As an entity's gauge reaches its completion threshold (value 100), the System allocates an operational Action Token to it.

UC-14 - Spawn Enemy

- **Primary Actor:** Game Clock
- **Description:** Periodically introduces fresh hostile actors along perimeter wall cells to preserve mechanical tension.
- **Pre-conditions:** Play Mode is active, and enemy counts fall below global density caps.
- **Success Guarantees (Post-conditions):**
    1. A new enemy entity is spawned at a selected peripheral edge boundary cell.
- **Normal Flow:**
    1. The Game Clock records a 9-second tracking interval threshold.
    2. The System parses outer boundary wall layouts to pinpoint unblocked, valid floor spawn locations.
    3. The System evaluates a probability distribution algorithm to determine entity configuration (e.g., values 1–60 instantiate a Knight; 61–90 instantiate a Sorcerer; 91–100 skip the spawn event).
    4. The System places the newly built entity into the active simulation layer and registers its attributes.

📊 Interface Management & Session Endings

UC-15 - Show HUD

- **Primary Actor:** System
- **Description:** Monitors character domain state modifications and updates the Heads-Up Display rendering values in real-time.
- **Pre-conditions:** The game session is actively running inside **Play Mode**.
- **Success Guarantees (Post-conditions):**
    1. Graphic HUD meters accurately portray active character attribute properties.
- **Normal Flow:**
    1. The System catches a modification signal targeting the Hero's state model variables (HP, Mana, STR, DEF, or Energy).
    2. The System calculates raw scaling factors, parses numeric strings, and rerenders the HUD resource bars and structural text indicators.

UC-16 - Pause / Resume Game

- **Primary Actor:** Player
- **Secondary Actor:** Game Clock
- **Description:** Temporarily suspends the runtime execution loop and safely restores execution on command.
- **Pre-conditions:** The application is handling an active **Play Mode** session.
- **Success Guarantees (Post-conditions):**
    1. Simulation states are frozen without risking data loss or state corruption.
- **Normal Flow:**
    1. The Player hits the ESC key or triggers the graphical pause button element.
    2. The System blocks the Game Clock mechanism (suspending AI processing, resource updates, and movement actions) and paints the Pause Menu layout over the viewport.
    3. The Player selects the **Resume** command item.

4. The System removes the graphical menu overlay and restarts the Game Clock tick loops.

UC-17 - Determine Winner & End Game

- **Primary Actor:** System
- **Description:** Evaluates the overarching victory or defeat states to safely terminate the gameplay sequence.
- **Pre-conditions:** An ultimate termination state condition is reached.
- **Success Guarantees (Post-conditions):**
    1. Main simulation tracking loops are safely disconnected, and final score/status view overlays are painted.
- **Normal Flow:**
    1. The System detects an absolute session endgame trigger condition.
    2. **Victory Path Evaluation:** Triggered when the Player secures the unique `VictoryCoin` dropped by the Final Boss on Level 3, opens their hotbar inventory array, and explicitly clicks to consume the coin item.
    3. **Defeat Path Evaluation:** Triggered automatically the moment the Hero entity's active HP balance falls to ≤ 0.
    4. The System stops the runtime clock, drops input control listening loops, calculates end-game runtime analytics, and switches views to the final results screen panel.

UC-18 - Game Over (Hero Defeat)

- **Primary Actor:** System
- **Description:** Coordinates death state routines when the primary character's health reserves dry up completely.
- **Pre-conditions:** The Hero entity's current HP falls to ≤ 0.
- **Success Guarantees (Post-conditions):**
    1. Input handlers are decoupled, and interaction options (**Restart Level** / **Main Menu**) are presented.
- **Normal Flow:**
    1. The System catches a condition update showing Hero HP ≤ 0.
    2. The System locks player input control arrays, freezes background clock cycles, and displays the **Game Over** interface panel overlay.
    3. The Player chooses either to click **Restart Level** (re-initializing the level architecture) or **Quit to Main Menu**.

💾 Data Persistence Operations

UC-19 - Save Game

- **Primary Actor:** Player
- **Secondary Actor:** File System
- **Description:** Serializes all active level architecture matrices, structural component coordinates, and persistent Hero attribute data into a structured JSON payload.
- **Pre-conditions:** The Player invokes the **Save** command execution.
- **Success Guarantees (Post-conditions):**
    1. A structured `.json` data file is generated and stored inside the dedicated storage directory (`saves/`).
- **Normal Flow:**
    1. The Player selects the **Save** option item from either the Design layout screen or the Pause Menu grid.
    2. The System compiles the grid layout specifications, active entities coordinates, current `currentLevel` trackers, Hero attribute metrics, and hotbar slots into a structured data transfer object.
    3. The File System writes this compiled text payload cleanly into a target `.json` file.

UC-20 - Load Game

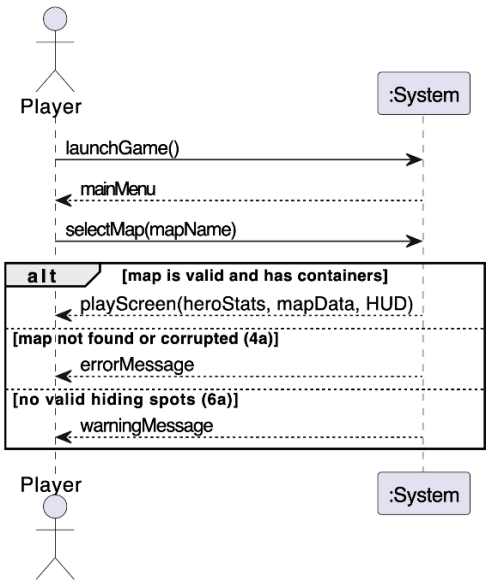
- **Primary Actor:** Player
- **Secondary Actor:** File System
- **Description:** Parses an existing JSON configuration file to restore map architectures and character attributes.
- **Pre-conditions:** The user triggers **Load Game**, and a valid `.json` save document exists within the file path.
- **Success Guarantees (Post-conditions):**
    1. Grid layouts and character models are re-instantiated seamlessly at their saved data state.
- **Normal Flow:**
    1. The Player enters the **Load Game** interface panel and confirms their choice of save state file.
    2. The File System accesses and streams the data content from the selected JSON document.
    3. The System purges all items from the active map scene, rebuilds structural tile variables, restores exact entity positioning, and updates the Hero's health and inventory arrays.

UC-21 - Start Game (Map Loading & Initialization)

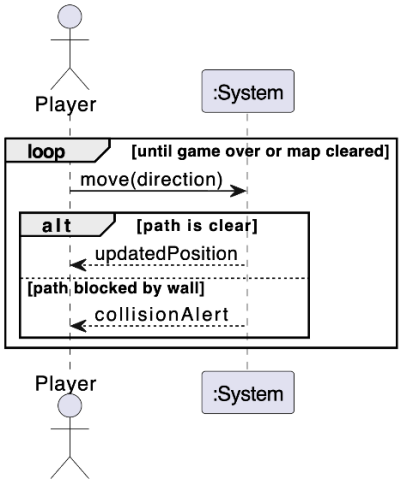
- **Primary Actor:** Player
- **Secondary Actor:** File System, Game Clock
- **Description:** Establishes core character attributes, populates level-specific container data arrays, and triggers the gameplay core loop execution.
- **Pre-conditions:** A map framework has been built or chosen from file configurations.
- **Success Guarantees (Post-conditions):**
    1. Map graphics are rendered, Hero stats are loaded, containers are filled with relevant items, and the session clock begins ticking.
- **Normal Flow:**
    1. The Player initiates play mode execution from the validation terminal screen.
    2. The System configures the Hero's core base statistics:
    3. $$\text{HP} = 17, \quad \text{Mana} = 80, \quad \text{DEF} = 2, \quad \text{Energy} = 100, \quad \text{STR} = \text{Random Range}[8, 15]$$
    4. The System populates the map's chests and crates with appropriate loot tables, including item tools, recovery consumables, or essential `LevelKey` nodes depending on the active level depth.
    5. The System transitions views to the primary Play Screen canvas layer and activates the core Game Clock.

System Sequence Diagrams:

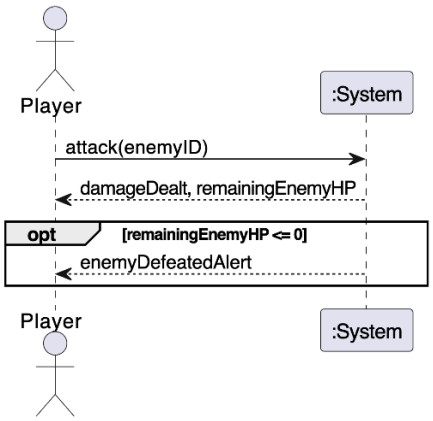
SSD-UC_01: Play Game



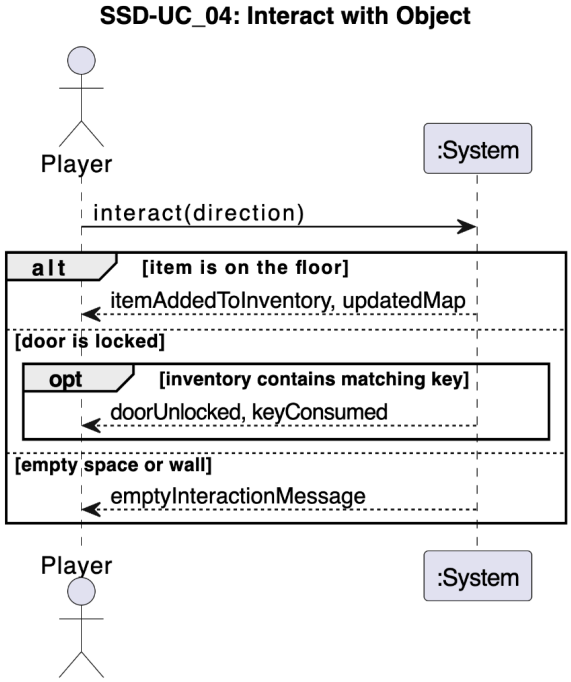
SSD-UC_02: Move Hero



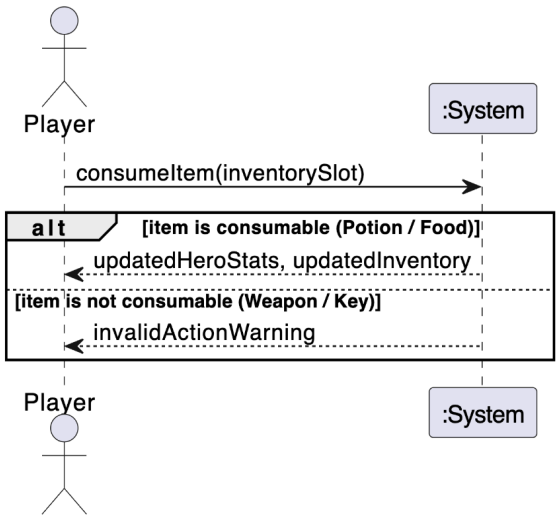
SSD-UC_03: Attack Enemy



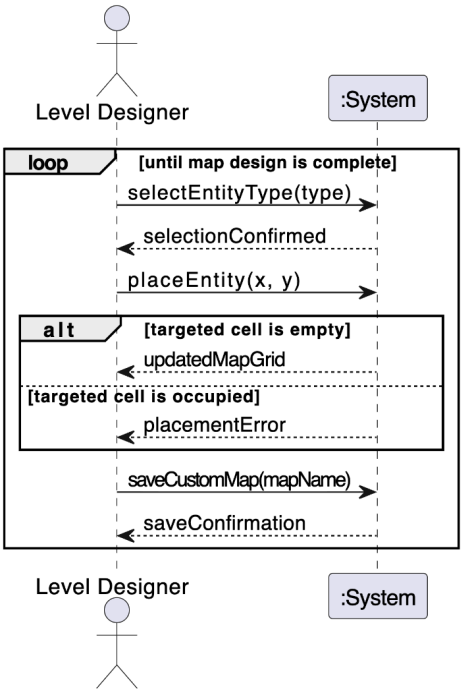
SSD-UC_04: Interact with Object



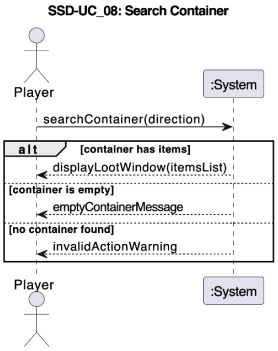
SSD-UC_05: Consume Item



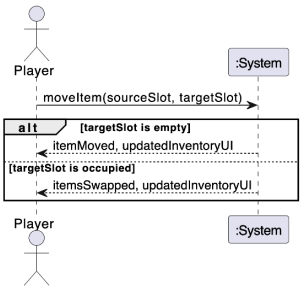
SSD-UC_06: Place Entity (Level Editor)



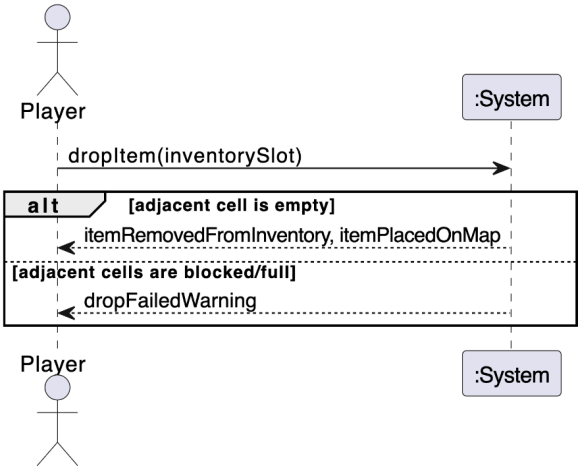
SSD-UC_08: Search Container



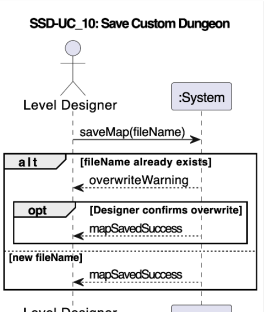
SSD-UC_09: Manage Inventory



SSD-UC_07: Drop Item



SSD-UC_10: Save Custom Dungeon



OPERATIONAL CONTRACTS

1. Move Hero

- **Operation:** `moveHero(direction: Direction)`
- **Cross References:** Player Movement, Escaping Enemies, Map Exploration.
- **Preconditions:**
  - Hero must be alive.
  - System must be in "Adventure" mode and inputs must be enabled.
  - `direction` must be a valid direction (W, A, S, D / Arrow Keys).
- **Postconditions:**
  - Hero was moved one tile in the target direction if no obstacle blocked the path (Hero coordinate attributes were modified).
  - Hero's animation state was set to the corresponding direction (e.g., `WALK_UP`, `WALK_DOWN`).
  - If the `shadowClone` is active and alive, its position was moved in the **exact opposite** direction of the Hero's movement.

2. Interact with Object

- **Operation:** `interact()`
- **Cross References:** Opening Doors, Picking Up Items, Opening Chests, Searching Walls.
- **Preconditions:**
  - Hero must be alive.
  - The user has pressed the 'E' key.
- **Postconditions:**
  - If there is an interactive object on the Hero's own tile, the appropriate action (e.g., entering a level door, picking up an item) was executed.
  - If not, the surrounding 3x3 tiles were scanned, and the most relevant interactive object in the Hero's facing direction was found.
  - If the selected object is a Door: If it is locked, the Inventory was searched for a Key; if found, the lock was unlocked, the door state was changed to "open", the key was destroyed, and the unlock `SoundEvent` was triggered.
  - If the selected object is a `SearchableObject`: A search popup dialog was displayed on the UI, and the action was executed based on the user's input.
  - If the object has multiple actions or is a Chest/Crate: A multiple-choice interaction dialog was displayed on the UI, and the corresponding action (`BreakAction`, `OpenAction`, `TakeAction`) was executed based on the player's choice.

3. Perform Attack

- **Operation:** `attack()`
- **Cross References:** Combat, Firing Projectiles, Damaging Enemies.
- **Preconditions:**
    - Hero must be alive.
    - The user has pressed the 'Space' key.
    - Hero's current Energy must be >= 10 (and sufficient Mana if using a Ranged weapon).
- **Postconditions:**
    - Hero's animation state was set to `ATTACK`.
    - Hero's Energy was reduced by 10.
    - **If the equipped weapon is Ranged:**
        - Hero's Mana was reduced according to the weapon's cost (accounting for Ring bonuses).
        - A new `Projectile` instance was created traveling in the Hero's facing direction with appropriate damage (Instance Creation).
        - The projectile was added to the `entities` list and a shoot sound was played.
    - **If the equipped weapon is Melee:**
        - The target tile in the Hero's facing direction was checked.
        - If a living enemy (Entity) was present on that tile, the Hero's attack function was executed on the target, reducing the target's HP. If no target was found, a swing sound was played.

4. Use Hotbar Item

- **Operation:** `useHotbarItem(slotIndex: int)`
- **Cross References:** Drinking Potions, Equipping Weapons/Armor, Reading Scrolls.
- **Preconditions:**
    - `slotIndex` must be a valid number between 1 and 8.
    - An item (`Item`) must exist in the Hero's inventory at that index.
- **Postconditions:**
    - One of the available actions for the item (`Use`, `Equip`, `Wear`, `Read`, or `Eat`) was executed.
    - If the item is wearable (Wearable), it was assigned to the equipped slot and the Hero's attack/defense stats were updated.
    - If the item is consumable (Usable/Potion), the Hero's Health, Mana, or Energy was increased, and the item instance was removed from the Inventory.
    - The selected hotbar slot was updated on the UI and the screen was repainted.

5. Discard Item

- **Operation:** `discardItem()`
- **Cross References:** Inventory Management.
- **Preconditions:**
  - The user has pressed the 'Q' key.
  - A specific item slot must be currently selected on the hotbar.
- **Postconditions:**
  - The `DiscardAction` defined on the item was executed.
  - The item was removed from the Hero's inventory.
  - The inventory UI was updated to reflect the removal.

6. Place Object (Design Mode)

- **Operation:** `placeObject(mouseX: int, mouseY: int)`
- **Cross References:** Map Creation, Design Mode, Limit Validation.
- **Preconditions:**
  - The game must be in "Design Mode".
  - The player must have selected a valid item type from the Palette.
  - The left mouse button must be clicked or dragged.
- **Postconditions:**
  - Mouse coordinates were converted to isometric/grid map coordinates (`hoverTileX`, `hoverTileY`).
  - If there wasn't already a "LevelDoor" at the target coordinate, the process continued.
  - Limit checks were performed based on the object type (e.g., `MAX_ITEMS`, `MAX_OBSTACLES`, `MAX_DECORATIVE_PER_MAP`). If limits were exceeded, the action was canceled and a warning dialog was shown to the user.
  - If limits were valid, a new instance was created using `GameObjectFactory` and saved to the `GameMap` at the specified coordinates.

7. Erase Object (Design Mode)

- **Operation:** `eraseObject(mouseX: int, mouseY: int)`
- **Cross References:** Correcting Mistakes in Design Mode.
- **Preconditions:**
  - The game must be in "Design Mode".
  - The right mouse button must be clicked.
- **Postconditions:**
  - The existing object at the corresponding tile was found.
  - If the object was a `LevelDoor`, deletion was blocked.

    - If the object was a wall (`WallTile`), its decoration and itself were reset and cleared from the map.
    - For other cases, a standard `FloorTile` was assigned to that tile, severing the link to the previous object.

8. Start Game

- **Operation:** `startGame(mode: GameMode)`
- **Cross References:** Main Menu, New Game, Transition from Design Mode to Play Mode.
- **Preconditions:**
    - System must be in the Main Menu or Design Mode.
    - A map (either default randomized or user-designed) must be ready.
- **Postconditions:**
    - A new Hero and Enemies (Knight, Sorcerer, etc.) were instantiated.
    - If the selected mode is "Adventure", a random Level Door and its hidden Level Key were placed on the map.
    - Game state (timers, logic) was initialized and the UI transitioned to `GameView`.
    - All old event listeners (EventBus) were cleared and new ones were registered.

9. Save Game

- **Operation:** `saveGame(fileName: String)`
- **Cross References:** Pause Menu, Saving Progress.
- **Preconditions:**
    - The game must be actively played (Adventure Mode) and currently paused.
- **Postconditions:**
    - The Hero's current state (Health, Mana, Inventory, Equipped Items, Coordinates) was read via `SaveManager`.
    - The current HP and positions of all enemies on the map were recorded.
    - The statuses of chests, searchable objects, and other map items were serialized.
    - All this information was encapsulated into a `GameState` instance and written to disk as a JSON file named `fileName`.
    - A success dialog was displayed on the UI.

10. Load Game

- **Operation:** `loadGame(state: GameState)`
- **Cross References:** Main Menu, Resuming Saved Game.
- **Preconditions:**
    - A valid saved `GameState` instance must have been read from disk.

- **Postconditions:**
    - A new map was instantiated and the Hero's stats and inventory were restored based on the `GameState` (Instance Creations).
    - Enemies, dropped items (Scrolls, Potions), doors (locked/open states), and flying projectiles were placed back onto the map according to their saved coordinates and statuses.
    - Spawner timers (for enemies/scrolls) were updated.
    - The game engine was started and the UI transitioned to `GameView`.

11. Save Map (Design Mode)

- **Operation:** `saveMap(filePath: String)`
- **Cross References:** Map Editor, Persistence.
- **Preconditions:**
    - The game must be in Design Mode.
    - The map must be valid according to `MapValidator` rules before saving.
- **Postconditions:**
    - Only static structures (Walls, Doors, Items, Obstacles) on the map were parsed and saved as a `.mapjson` file.
    - A success dialog was presented to the user.

12. Load Map (Design Mode)

- **Operation:** `loadMap(filePath: String)`
- **Cross References:** Map Editor, Modding.
- **Preconditions:**
    - The game must be in Design Mode.
    - A valid `.mapjson` file must be selected.
- **Postconditions:**
    - The structures within the `.mapjson` file were read, instantiated from scratch via `GameObjectFactory`, and populated onto the Design Mode grid.

13. Toggle Pause

- **Operation:** `togglePause()`
- **Cross References:** Game Flow Control.
- **Preconditions:**
    - The game (`GameView`) must be active and focused.
    - The user has pressed the `ESC` key.
- **Postconditions:**
    - The game loops (Logic Timer, Render Timer) and spawner timers were stopped.

    - A semi-transparent `PauseMenu` was overlaid on the screen (GlassPane).
    - When pressed again, the menu was hidden and the timers were restarted from where they left off.

14. Advance Level

- **Operation:** `advanceLevel()`
- **Cross References:** Level Progression, Dungeon Depth.
- **Preconditions:**
    - Hero must be alive.
    - Hero successfully interacts with an unlocked `LevelDoor`.
- **Postconditions:**
    - The game timers were temporarily frozen.
    - `LevelManager` generated or loaded a brand new map layout for the next depth.
    - The Hero was relocated to the starting position (e.g., coordinate 2,2) on the new map.
    - All existing enemies and map items from the previous level were destroyed.
    - New enemies and items appropriate for the new level were spawned and populated.
    - A "Level X" floating text was displayed, and game timers were resumed.

15. Toggle Inventory Panel

- **Operation:** `toggleInventory()`
- **Cross References:** Inventory Management, UI Interaction.
- **Preconditions:**
    - The game (`GameView`) must be active and focused.
    - The user has pressed the `I` key.
- **Postconditions:**
    - The visibility boolean of the expanded inventory panel was flipped (from hidden to visible or vice-versa).
    - The `GameView` was instructed to repaint itself, rendering the inventory UI overlay if visible, or removing it if hidden.

VISION DOCUMENT: DUNGEON CRAWLER

1. Introduction

1.1 Purpose

The purpose of this Vision Document is to collect, analyze, and define the high-level needs and features of the Dungeon Crawler project. It focuses on the capabilities required by stakeholders and target users, providing a clear roadmap of what the software will do and why it is being built.

1.2 Scope

This project is a 2D tile-based Role-Playing Game (RPG) built using Java. It features grid-based exploration, real-time combat, inventory management, and a robust custom map editor. The system is designed using Object-Oriented Analysis and Design (OOAD) principles, emphasizing low coupling, high cohesion, and standard design patterns.

1.3 Definitions, Acronyms, and Abbreviations

- **GUI: Graphical User Interface**
- **OOAD: Object-Oriented Analysis and Design**
- **GRASP: General Responsibility Assignment Software Patterns**
- **GoF: Gang of Four (Design Patterns)**
- **RPG: Role-Playing Game**
- **FURPS+: Functionality, Usability, Reliability, Performance, Supportability**

2. Positioning

2.1 Business Opportunity

The market for lightweight, moddable 2D dungeon crawlers is strong among casual and indie game fans. However, many games lack built-in tools for players to easily design and share their own levels. Providing an integrated level editor within the game executable creates a higher replay value and community engagement opportunity.

2.2 Problem Statement

The problem of limited replayability in lightweight 2D dungeon crawlers affects casual gamers and creative players. The impact of this is a short product lifespan and a lack of community engagement. A successful solution would be to provide an integrated toolset that allows players to easily design, play, and share their own custom levels without needing external modding knowledge.

2.3 Product Position Statement

For gamers and creative level designers who enjoy classic rogue-lite RPG mechanics, the Dungeon Crawler is a desktop game that provides both an engaging adventure experience and an integrated Design Mode. Unlike traditional static games, our product empowers users to build and test custom dungeons seamlessly from within the main application.

3. Stakeholder and User Descriptions

3.1 Market Demographics

The target audience consists of casual desktop gamers who enjoy puzzle-solving and combat, as well as creative users who enjoy sandbox-style map building (similar to *Super Mario Maker*).

3.2 Stakeholder Summary

Stakeholder	Description
Players / Gamers	**The primary end-users who play the game for entertainment. They expect smooth controls, engaging combat, and an intuitive user interface.**
Level Designers	**Creative users who utilize the Design Mode to craft intricate puzzles, enemy encounters, and custom maps to challenge themselves or others.**

Developers (Students)	**The engineering team (COMP302 students) responsible for designing the architecture, implementing design patterns, and writing maintainable code.**
Instructors / Evaluators	**University professors and Teaching Assistants who will evaluate the project based on its adherence to software engineering principles (GRASP, GoF Patterns, SOLID).**

3.3 User Environment

The application runs as a local desktop client on Windows, macOS, or Linux via the Java Virtual Machine. Users interact with the system using a keyboard and mouse. The primary interface is a 1250x800 graphical window.

4. Product Overview

4.1 Product Perspective

The Dungeon Crawler is a standalone, self-contained desktop application built using Java and Swing. It does not require internet connectivity. Game saves (.json) and custom maps (.mapjson) are serialized and stored locally on the user's filesystem.

4.2 Summary of Capabilities

- **Procedural and custom dungeon exploration.**
- **Real-time action-oriented combat (melee and ranged).**
- **Inventory and resource management.**
- **Complete drag-and-drop level editing suite.**
- **Game state serialization (Save/Load functionality).**

4.3 Assumptions and Dependencies

- **System Requirements: The user must have a Java Runtime Environment (JRE) installed to execute the application.**

- **Hardware: The game assumes the availability of a standard keyboard (for movement/actions) and a mouse (for inventory management and map design).**

5. Product Features (Key Features)

1. **Adventure Mode (Play Mode): A rogue-lite survival mode where players navigate through levels, manage resources (Health, Mana, Energy), fight diverse enemies (Knights, Sorcerers, Final Boss), and progress through Level Doors.**
2. **Team Match Mode: A secondary gameplay mode allowing teams of AI and players to fight against each other on a dynamically split map.**
3. **Design Mode (Map Editor): A comprehensive drag-and-drop level editor. Users can select tiles, enemies, items, and interactive objects from a palette to create custom maps, complete with validation limits (e.g., maximum items or obstacles).**
4. **Dynamic Combat System: Real-time grid-based combat featuring both melee weapon strikes and ranged magical projectiles.**
5. **Interactive Environments: High environmental interactivity including doors requiring specific keys, breakable crates, lockable chests, and searchable walls that may contain hidden treasures or trigger deadly traps.**
6. **Inventory & Hotbar Management: A dynamic inventory UI overlay equipped with an 8-slot hotbar for quick access. Players can seamlessly equip armor/weapons, drink potions, or read scrolls (e.g., Shadow Clone scroll).**
7. **Save & Load System: Full serialization of the GameState, allowing players to pause and save their exact progress—including enemy locations, map modifications, and inventory contents—and resume at a later time.**

6. Constraints

- **Platform Constraints: Must be executable on any machine that supports Java (*Write Once, Run Anywhere*).**
- **Architectural Constraints: Must be built entirely in pure Java using Swing/AWT for graphics. No external game engines (like Unity, Godot, or LibDX) are permitted.**

- **Design Constraints: The architecture must strictly adhere to GRASP (General Responsibility Assignment Software Patterns) and GoF (Gang of Four) Design Patterns. Anticipated patterns include *Factory* (for object creation), *Prototype* (for cloning map objects), *Observer* / *EventBus* (for decoupling audio/visual events from logic), and *State* (for character animations).**

7. Quality Ranges (FURPS+)

7.1 Functionality

The game must correctly calculate combat damage, enforce collision detection (walls, obstacles, map borders), and accurately validate custom maps during the saving process.

7.2 Usability

The interface must be highly intuitive. The Design Mode must allow dragging and dropping without requiring technical knowledge. Tooltips and a Help Dialog should guide the user.

7.3 Reliability

The application should not crash during gameplay. If an invalid map is loaded, the game must gracefully handle the error and notify the user via a popup dialog rather than terminating the process.

7.4 Performance

The game must maintain a smooth, responsive frame rate via its internal logic (approx. 60 ticks per second) and render timers. Saving and loading operations should execute within a few seconds without causing the application to freeze.

7.5 Supportability

The codebase must be heavily documented and conform to standard Object-Oriented principles, allowing future developers to easily add new enemies, items, or tiles by simply extending existing base classes.

SUPPLEMENTARY SPECIFICATION: DUNGEON CRAWLER

1. Introduction

1.1 Purpose

This document captures the system requirements that are not readily captured in the use cases of the Dungeon Crawler project. This includes non-functional requirements (such as performance, usability, and reliability), design constraints, and overarching domain rules. It supplements the Use Case Model and the Vision Document to provide a complete picture of the system's requirements.

1.2 Scope

These specifications apply to the entire Dungeon Crawler Java application, including both the "Adventure" gameplay mode and the "Design" mode (Map Editor).

2. Functionality

2.1 Error Handling & Logging

- **Asset Loading: If an image or sound file (e.g., in resources/images/ or resources/sounds/) is missing, the system shall catch the IOException, print a warning to the standard error output (System.err), and fall back to drawing a colored rectangle or a placeholder default texture. The game shall not crash.**
- **Map Validation Errors: When loading a custom .mapjson file in Design Mode, if the file is corrupted or contains unrecognized objects, the system must abort the load, revert to the previous or default map state, and display a user-friendly InvalidMapDialog rather than terminating the process.**

2.2 Security

- **Local Sandbox: The application shall not require internet access, nor will it attempt to execute remote code. It shall only read and write .json and .mapjson files to the designated local saves/ directory.**

3. Usability

3.1 Key Bindings and Controls

- **The game shall be fully controllable using a standard QWERTY keyboard and a mouse.**
- **Movement: W, A, S, D or Arrow Keys.**
- **Interaction: E key (Context-sensitive interaction with doors, chests, or items).**
- **Combat: Spacebar (Executes an attack based on the currently equipped weapon).**
- **Inventory Management:**
  - **Hotbar slots must be bound to number keys 1 through 8.**
  - **Pressing Q will discard the item currently selected on the hotbar.**
  - **Pressing I will toggle the full expanded inventory view.**
- **System: ESC will pause the game and display the Pause Menu.**

3.2 Map Editor Ergonomics

- **The Design Mode must support "Drag and Drop" (mouse drag) continuous placement of floor tiles and walls.**
- **The Right Mouse Button (RMB) shall act universally as an "Eraser" tool to remove objects or reset walls to standard floor tiles.**

4. Reliability

4.1 Recoverability

- **If the game application is closed unexpectedly (e.g., power failure), the save file mechanism must ensure that it writes to a temporary file first before overwriting the main .json save file to prevent total save corruption. (Ideal target; currently mitigated by manual saving).**

4.2 Predictability

- **Procedural level generation must rely on fixed seed mechanics if repeatable testing is required. Otherwise, it must consistently produce solvable maps (i.e., Level Doors must always have a reachable Level Key).**

5. Performance

5.1 Response Time and Frame Rate

- **The main gameplay loop (Logic Timer) shall operate at roughly 60 ticks per second (approx. 16ms delay).**
- **The rendering loop (Render Timer) shall also attempt to draw at 60 FPS, ensuring smooth sprite animations and projectile travel without visible stutter on modern standard hardware.**

5.2 Resource Usage

- **The application's memory footprint must remain below 512 MB of RAM during typical gameplay.**
- **Images and sprites should be cached in an AssetManager using the Flyweight/Singleton pattern to prevent redundant memory allocation when instantiating hundreds of FloorTile or WallTile objects.**

6. Supportability

6.1 Coding Standards

- **All code must be written in Java.**
- **The system must follow strict Model-View-Controller (MVC) or Model-View separation. Domain classes (e.g., Hero, Knight) must not import javax.swing or java.awt rendering classes directly.**
- **Feedback to the UI (such as playing a sound or drawing floating damage text) must be handled asynchronously via a GameEventBus (Observer Pattern).**

6.2 Extensibility

- **Adding a new Enemy or Item should not require modifying existing core engine logic. New entities should simply extend the Entity or Item base classes and register their specific stats or animations.**

7. Design Constraints

- **Framework Constraint: The Graphical User Interface must be built using pure Java Swing and AWT. The use of external Java game libraries (e.g., LibGDX, LWJGL) or external engines is strictly prohibited by the project rubric.**

- **Audio Format: Sound effects must be compatible with the standard javax.sound.sampled.Clip library (preferably .wav format).**

8. Domain Rules (Business Rules)

Rule ID	Description
DR-01	Action Costs: A melee attack always consumes exactly 10 Energy. A ranged attack consumes Mana equal to the weapon's base requirement minus any equipped Ring bonuses, plus 10 Energy. If the Hero lacks sufficient Energy or Mana, the attack action is blocked.
DR-02	Map Limitations: A map in Design mode cannot contain more than MAX_ITEMS consumable/wearable items, or MAX_OBSTACLES interactive blockages (crates/chests) to ensure balance.
DR-03	Indestructible Objects: A LevelDoor (the exit to the next floor) is indestructible and cannot be erased by the player in Design Mode once placed programmatically.
DR-04	Inventory Capacity: The Hero's hotbar is strictly limited to 8 slots. Picking up an item when all 8 slots are full will result in the item being rejected or swapped.

DR-05	**Shadow Clone Logic: The Shadow Clone must mimic the player's movement in the *exact reverse* direction. It possesses a distinct lifespan (timer) and is instantly destroyed if its HP reaches 0.**

GLOSSARY: DUNGEON CRAWLER

1. Introduction

1.1 Purpose

This Glossary provides a highly comprehensive definition of the core terminology, domain concepts, technical jargon, and sub-systems used throughout the **Dungeon Crawler** project. Its purpose is to ensure clear, unambiguous communication among all stakeholders, acting as a complete Data Dictionary for the Domain Model.

2. Game Modes & Core Concepts

Term	Definition	Aliases / Related Terms
Adventure Mode	The primary rogue-lite survival mode where the player explores procedural or pre-made maps, fights enemies, and descends into deeper levels.	Play Mode, Survival Mode
Design Mode	The integrated map editor state that allows users to visually drag-and-drop tiles, enemies, and items to create custom .mapjson levels.	Map Editor, Level Editor
Team Match Mode	A secondary gameplay mode dividing the map into halves, where teams of AI (e.g., Orange vs. Cyan) fight against each other.	Skirmish Mode

Level	A distinct floor or depth within the dungeon. The player transitions between levels by finding and entering a Level Door.	Depth, Floor
Grid	The invisible coordinate system (e.g., 32x32 pixels per square) that dictates movement, collision, and object placement in the game world.	Coordinate System

3. Entities & Characters

Term	Definition	Aliases / Related Terms
Entity	The base abstract class for a living, moving character on the grid that possesses Health Points (HP) and can engage in combat.	Actor, Character
Hero	The primary protagonist and player-controlled Entity. The Hero manages resources like HP, Mana, and Energy, and possesses an Inventory.	Player, Main Character
Knight	A basic melee-focused enemy Entity that attempts to close the distance and strike the Hero.	Melee Enemy
Sorcerer	A magical enemy Entity that can teleport around the map and fire ranged projectiles at the Hero.	Ranged Enemy
Final Boss	The ultimate enemy Entity encountered at the deepest level. It possesses distinct phases (e.g., at 80%, 60%, 40% HP) that alter its attack patterns.	Boss

Shadow Clone	A temporary magical Entity spawned by reading a scroll. It mimics the Hero's movements in the exact *reverse* direction.	Clone, Mirror Image

4. Resources & Attributes

Term	Definition	Aliases / Related Terms
Health Points (HP)	The life force of an Entity. If the Hero's HP reaches 0, it results in a Game Over.	Health, Life
Energy	A physical stamina resource utilized by the Hero. Each physical melee attack consumes exactly 10 Energy.	Stamina
Mana	A magical resource utilized by the Hero to fire ranged projectiles (e.g., using a Fire Wand).	Magic Points, MP
Strength (Str)	A base attribute that scales the physical damage dealt by the Hero's melee attacks.	Attack Power

5. Items & Inventory

Term	Definition	Aliases / Related Terms
Inventory	The underlying data structure holding all items collected by the Hero.	Bag, Backpack
Hotbar	A specialized UI element containing 8 slots that allows the player to quickly access and use items via number keys (1-8).	Quickbar, Action Bar
Usable Item	An item that is consumed upon use to provide an immediate effect (e.g., Health Potion, Mana Potion, Energy Potion).	Consumable
Wearable Item	An item that can be equipped to alter the Hero's stats permanently while worn (e.g., Swords, Armor, Rings).	Equipment, Gear
Key Item	A standard key used to unlock static wooden/iron doors and regular chests. It is destroyed upon use.	Regular Key

Level Key	A special, indestructible key specifically required to unlock the Level Door to advance to the next floor.	Boss Key, Golden Key
Victory Coin	The ultimate objective item. Collecting this coin immediately triggers the "You Win" victory sequence.	MacGuffin
Projectile	A moving physical or magical entity fired by a ranged weapon or enemy. It travels in a straight line and deals damage upon collision.	Magic Missile, Arrow

6. Environment & Static Objects

Term	Definition	Aliases / Related Terms
GameObject	The absolute base architectural class for anything that exists on the map grid. Everything inherits from this.	Map Object
FloorTile	A walkable tile that forms the ground of the dungeon. Entities and items can freely exist on top of it.	Ground
WallTile	A solid block that prevents movement and blocks projectiles. It can hold a Decoration.	Obstacle

Door	A static object blocking a pathway. It can be open, closed, or locked (requiring a Key Item).	Wooden Door
Level Door	An indestructible door serving as the exit to the current level. Often locked, requiring a Level Key.	Exit Gate
Chest	A container that may hold loot. It can be locked, requiring a key, or unlocked for immediate opening.	Treasure Chest
Crate	A fragile wooden box that must be broken (using 10 Energy) to reveal potential hidden items.	Barrel, Box
Searchable Object	A visual decoration on a WallTile (e.g., a painting or crack) that the player can inspect. It may contain loot or trigger a trap.	Secret Wall
Decoration	A purely visual, non-interactive object (e.g., a Torch or Column) that adds atmosphere to the dungeon without blocking movement.	Prop

7. Systems & Managers (Architecture)

Term	Format / Structure	Description
GameMap	Data Structure	The 2D array matrix that manages spatial coordinates, bounds checking, and collision logic for all GameObjects.

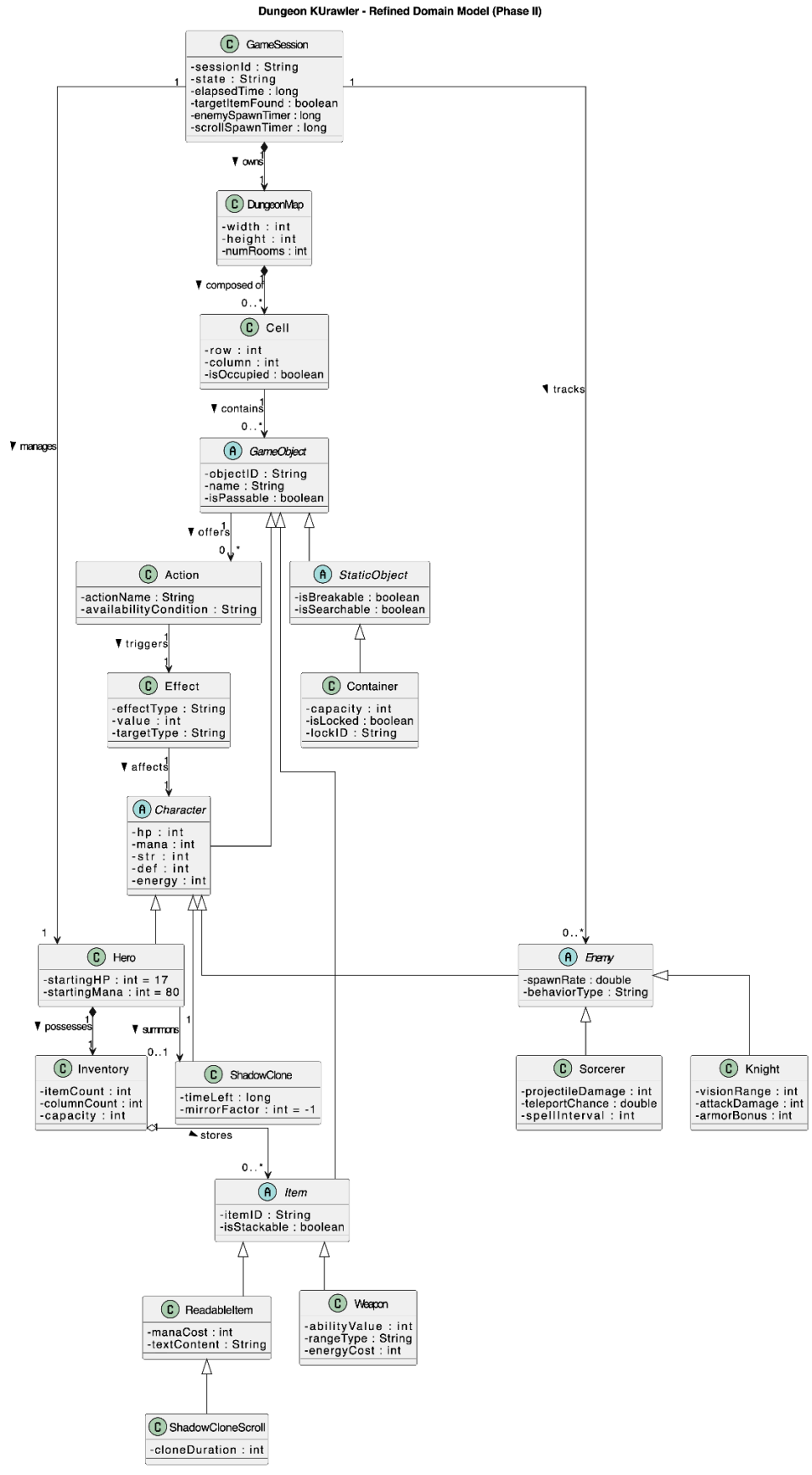
GameState	Serialized Object	A snapshot of the active session (Hero stats, enemies, map objects) converted to JSON format for the Save/Load feature.
.mapjson	File Format	The proprietary file extension used to save custom maps created in Design Mode.
LevelManager	Controller	Handles the generation of procedurally placed level doors, keys, and the mathematical scaling of enemies as depths increase.
SaveManager	Controller	Manages the I/O operations of writing GameState snapshots to the disk and parsing them back into memory.
MapValidator	Utility	An algorithm used in Design Mode to verify if a custom map meets required rules (e.g., has walkable paths) before allowing a save.
AssetManager	Singleton	A central registry that caches images and sprites to prevent redundant memory loading (Flyweight Pattern).
SoundManager	Utility	A class utilizing standard Java Audio clips to play .wav files based on triggered events.
EnemySpawner	Logic Timer	An algorithmic system that dynamically injects new enemies into the map at predefined intervals.

ScrollSpawner	Logic Timer	Similar to the EnemySpawner, but randomly drops beneficial magical scrolls onto the map over time.
PaletteManager	UI Controller	Manages the selection of tools and items in the left-hand panel during Design Mode.

8. Software Patterns & Terminology

- **Action (Command Pattern):** An interface defining executable verbs (e.g., TakeAction, OpenAction, BreakAction). Every GameObject exposes a list of valid Actions.
- **EventBus (Observer Pattern):** A publish-subscribe messaging system used to broadcast events (like sounds or floating damage text) without tightly coupling domain logic to rendering classes.
- **AnimationState (State Pattern):** An enum/state tracker that dictates which sprite sheet row should be drawn for a character (e.g., WALK_UP, ATTACK, IDLE).
- **GameObjectFactory (Factory Pattern):** A central class responsible for instantiating concrete game objects based on string identifiers, heavily used during map loading and Design Mode.
- **MVC (Model-View-Controller):** The core architectural separation ensuring that domain models (Hero, GameMap) know nothing about graphical rendering (GameView, JPanel).

Domain Model:



Dungeon KUrawler - Domain Model (Revised & Integrated)

This document describes the structural Domain Model of the Dungeon KUrawler application. It details the core Java classes, attribute boundaries, object inheritance hierarchies, and relational associations reflecting the final production code base.

💾 1. Core Session & World Architecture

GameState (Session State)

- **Description:** Holds the complete runtime snapshot of the active play session, enabling clean serialization/deserialization workflows for data persistence (saving and loading games).
- **Attributes:**
  - `saveName` *(String)* — Unique identifier for the saved profile.
  - `timestamp` *(String)* — Temporal stamp marking the save completion point.
  - `enemySpawnTimeLeft` *(long)* — Remaining tick counts before the next perimeter enemy wave triggers.
  - `scrollSpawnTimeLeft` *(long)* — Internal tracking gauge for consumable scroll resource generation.
  - `elapsedSeconds` *(long)* — Total gameplay runtime tracking variable.
  - `currentLevel` *(int)* — Active depth counter restricted strictly from **Level 1 to Level 3**.
- **Associations:**
  - Aggregates exactly **one** persistent `Hero` instance.
  - Maintains a tracking collection of active `Enemy` instances.
  - References active collections of active `MapItem` objects scattered across the map.
  - Manages active tracking registers for traveling flight `Projectile` objects.

GameMap & Grid

- **Description:** Represents the absolute spatial layout of the active dungeon tier depth. `GameMap` structurally extends the foundational `Grid` layer.
- **Attributes:**
  - `rows` *(int)* — Matrix width boundary fixed at **28 tiles**.
  - `cols` *(int)* — Matrix height boundary fixed at **20 tiles**.
- **Associations:**
  - Composed of a detailed 2D grid matrix of `GameObject` elements mapping individual spatial coordinates.
  - Instantiates static coordinates mapped explicitly into atomic `WallTile` or `FloorTile` node references.

⚔️ 2. Core Entities Hierarchy (Characters)

GameObject (Abstract Class)

- **Description:** The root superclass for all structural elements, interactive props, dynamic items, and living characters instantiated on the dungeon grid array.
- **Attributes:**
  - `name` *(String)* — Textual moniker for diagnostic tracking and HUD rendering.
  - `x` *(int)* — Zero-indexed column position coordinate on the map grid.
  - `y` *(int)* — Zero-indexed row position coordinate on the map grid.
  - `imageName` *(String)* — Resource target reference pointing to the corresponding UI asset filename.
  - `passable` *(Boolean)* — Computational gate variable defining collision behaviors for moving objects.
  - `customScale` *(double)* — Geometric scaling multiplier utilized during canvas draw operations.
- **Associations:**
  - Linked structurally with zero or more contextual execution `Action` profiles.

Entity (Abstract Class)

- **Description:** Represents dynamic, living characters on the grid layout capable of tracking combat metrics and processing interactive health damage behaviors.
- **Associations:**
  - Direct structural inheritance from the base `GameObject` class.

Hero

- **Description:** The primary player-controlled actor who navigates dungeon depths, manages equipment hotbars, combats hostiles, and aims to consume the Victory Coin.
- **Attributes:**
  - `hp` *(int)* — Current hit point pools calculated against a fixed initial **maximum cap of 17**.
  - `mana` *(int)* — Magic execution currency initialized to a **base ceiling of 80**.
  - `str` *(int)* — Core offensive strength modifier determined via a **random range of [8, 15]** at initialization.
  - `def` *(int)* — Physical defensive protection rating initialized to a **base standard value of 2**.
  - `energy` *(int)* — Action expenditure currency initialized to a **base standard ceiling of 100**.
  - `currentDirection` *(Direction)* — Enumerated token mapping active movement or tracking alignment states.
  - `currentAnimationState` *(AnimationState)* — Active animation vector regulating sprite canvas draw updates.

- **Associations:**
    - Inherits directly from the `Entity` abstraction layer.
    - Owns exactly **one** persistent `Inventory` management module (Aggregation).
    - Direct equipment association binding with zero or one active `equippedWeapon` instance.
    - Direct equipment association binding with zero or one active `equippedArmor` instance.
    - Direct equipment association binding with zero or one active `equippedRing` instance.

Enemy (Abstract Class)

- **Description:** The structural foundation for all hostile entities that instantiate procedurally along outer map boundaries to enforce ambient game challenge curves.
- **Associations:**
    - Direct structural inheritance from the `Entity` abstraction layer.

Knight (Subclass)

- **Description:** A standard melee-focused combatant that executes simple patrol pathings when isolated, switching to optimal pursuit vectors when closing distances to the player.
- **Attributes:**
    - Melee sweep actions delivering a fixed output profile of **4 HP base damage**.
- **Associations:**
    - Direct structural inheritance from the `Enemy` base class.

Sorcerer (Subclass)

- **Description:** A ranged magic-wielding hostile that runs random automated 7-second interval teleport sequences and fires linear projectile matrices.
- **Attributes:**
    - Magic spell actions casting traveling projectiles delivering a fixed output profile of **8 HP base damage**.
- **Associations:**
    - Direct structural inheritance from the `Enemy` base class.
    - Periodically instantiates dynamic `Projectile` instances onto the active world matrix.

FinalBoss / Hakan Ayral (Subclass)

- **Description:** An ultimate 2x2 tile cluster boss entity restricting its presence to the Level 3 arena. Operates on adaptive combat mechanics and triggers defensive minion waves at designated health intervals.
- **Associations:**
    - Direct structural inheritance from the `Enemy` base class.
    - Drops the mission-critical unique `VictoryCoin` immediately upon entity death.

ShadowClone (Subclass)

- **Description:** A temporary, allied unit summoned via scroll activation mechanics designed to assist the primary Hero in distracting hostile forces.
- **Associations:**
    - Direct structural inheritance from the `Entity` base class.

🎒 3. Inventory & Item Systems

Inventory & InventoryHotbar

- **Description:** Dictates storage bounds for item retention up to a strict capacity limit. The hotbar controller facilitates item activation mappings and handles select scroll utilities.
- **Attributes:**
  - `capacity` *(int)* — Absolute maximum space restriction fixed immutably at **8 available slots**.
  - `selectedSlot` *(int)* — Pointer mapping the active operational hotbar index bounding from **1 to 8**.
- **Associations:**
  - Attached as an integral functional component belonging to exactly **one** `Hero`.
  - Aggregates zero or more concrete `MapItem` object definitions.

MapItem (Abstract Class)

- **Description:** Concrete foundational class specifying any physical object entity eligible to be stored within hotbar arrays or dropped as loot items onto floor tiles.
- **Associations:**
  - Direct structural inheritance from the `GameObject` base class.

Weapon / Wearable Item (Subclass)

- **Description:** Equippable tool implementations (e.g., `SwordItem`, `AxeItem`, `BowItem`, `FireWandItem`, `KnightHammerItem`) that modify the player's primary combat damage output and attack delay variables.
- **Attributes:**
  - Granular weapon attack ratings and dynamic swing speed delays.
- **Associations:**
  - Direct structural inheritance from the `MapItem` abstraction layer.

ArmorItem (Subclass)

- **Description:** Wearable body gear pieces engineered to provide robust resilience against incoming hostile attacks.
- **Attributes:**
  - `defBonus` *(int)* — Static value appended directly onto the player's core defensive tracking rating.
- **Associations:**
  - Direct structural inheritance from the `MapItem` abstraction layer.

RingItem / Ring (Subclass)

- **Description:** Equippable accessory gear variants (such as `RedRing`, `GreenRing`, or `BlueRing`) granting passive boost modifiers to core HP, Strength, or Energy stats.
- **Associations:**
  - Direct structural inheritance from the `MapItem` abstraction layer.

PotionItem (Subclass)

- **Description:** Consumable utility items (including `HealthPotion`, `ManaPotion`, and `EnergyPotion`) engineered to instantly replenish diminished player resource pools.
- **Associations:**
  - Direct structural inheritance from the `MapItem` abstraction layer.

ShadowCloneScroll (Subclass)

- **Description:** A highly specialized magic scroll item that consumes a hotbar slot to materialize a friendly duplicate unit on the map.
- **Associations:**
    - Direct structural inheritance from the `MapItem` abstraction layer.

LevelKey (Subclass)

- **Description:** A critical progression-gated item key mandatory for overriding locks on regional LevelDoors to shift depths.
- **Associations:**
    - Direct structural inheritance from the `MapItem` abstraction layer.

KeyItem (Subclass)

- **Description:** A utility key item gathered to grant manual access to locked static Chest props encountered on exploration pathways.
- **Associations:**
    - Direct structural inheritance from the `MapItem` abstraction layer.

VictoryCoin (Subclass)

- **Description:** The ultimate objective item dropped exclusively upon defeating the Final Boss. Activating it from the hotbar triggers absolute victory conditions.
- **Associations:**
    - Direct structural inheritance from the `MapItem` abstraction layer.

🧱 4. Static Objects & Environmental Elements

StaticObject (Abstract Class)

- **Description:** Superclass detailing non-character structural obstacles, breakable barricades, or interactive containers placed throughout map generation layers.
- **Associations:**
    - Direct structural inheritance from the foundational `GameObject` class.

Column (Subclass)

- **Description:** Impassable architectural columns utilized to block movement and structure exploration channels.
- **Associations:**
    - Direct structural inheritance from the `StaticObject` abstraction layer.

Crate / DoubleCrate (Subclass)

- **Description:** Fragile wooden containers built to be searched for simple items or shattered entirely via structural weapon swings.
- **Associations:**
    - Direct structural inheritance from the `StaticObject` abstraction layer.

Chest (Subclass)

- **Description:** Standard storage chests containing valuable item drop tables. Can be generated in locked configurations requiring an active `KeyItem`.
- **Attributes:**
    - `isLocked` *(Boolean)* — Structural state gate managing interaction validation checks.

- **Associations:**
    - Direct structural inheritance from the StaticObject abstraction layer.

SearchableObject / WallObject (Subclass)

- **Description:** Decorative elements mounted to wall cells (e.g., Torches, Flags, Statues, Cobwebs, Chains) that players can actively search to attempt minor item discovery.
- **Attributes:**
    - isSearched *(Boolean)* — Tracks whether the specific node has been picked over to block duplicate collection exploits.
- **Associations:**
    - Direct structural inheritance from the StaticObject abstraction layer.

Door (Subclass)

- **Description:** Spatial node gating structures that block movement vectors entirely until a valid interaction unlocks or shifts them open.
- **Attributes:**
    - isLocked *(Boolean)* — Tracking state regulating doorway access.
- **Associations:**
    - Direct structural inheritance from the StaticObject abstraction layer.

LevelDoor (Subclass)

- **Description:** A massive, triple-scaled special exit door structurally anchored at exit coordinate location (11, 0). Requires a matching LevelKey to initiate world level clear actions.
- **Associations:**
    - Direct structural inheritance from the parent Door subclass implementation.

⚙️ 5. Behavioral Mechanics

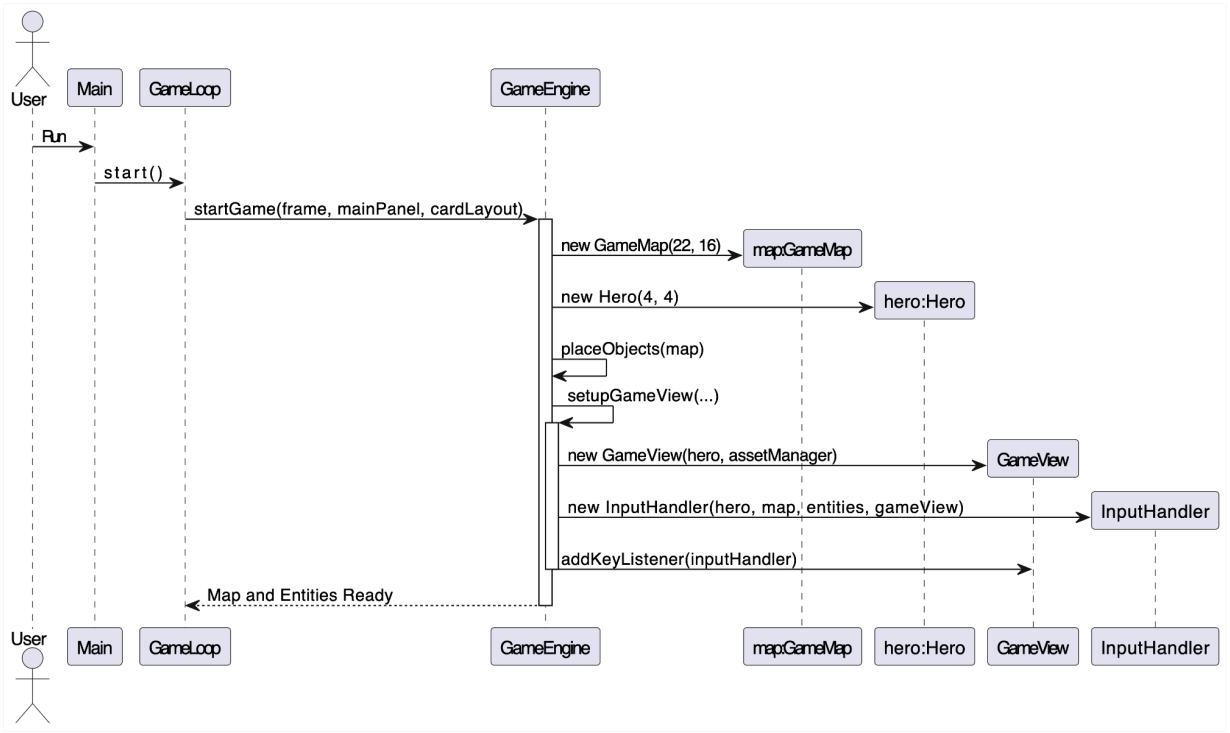
Action & Effect

- **Description:** Encapsulates the explicit execution patterns triggered when the player coordinates physical interactions against environmental GameObjects. Examples include:
    - SearchAction
    - BreakAction
    - OpenAction
    - TakeAction
    - DiscardAction
    - EatAction
    - EquipAction
    - UnlockLevelGateAction

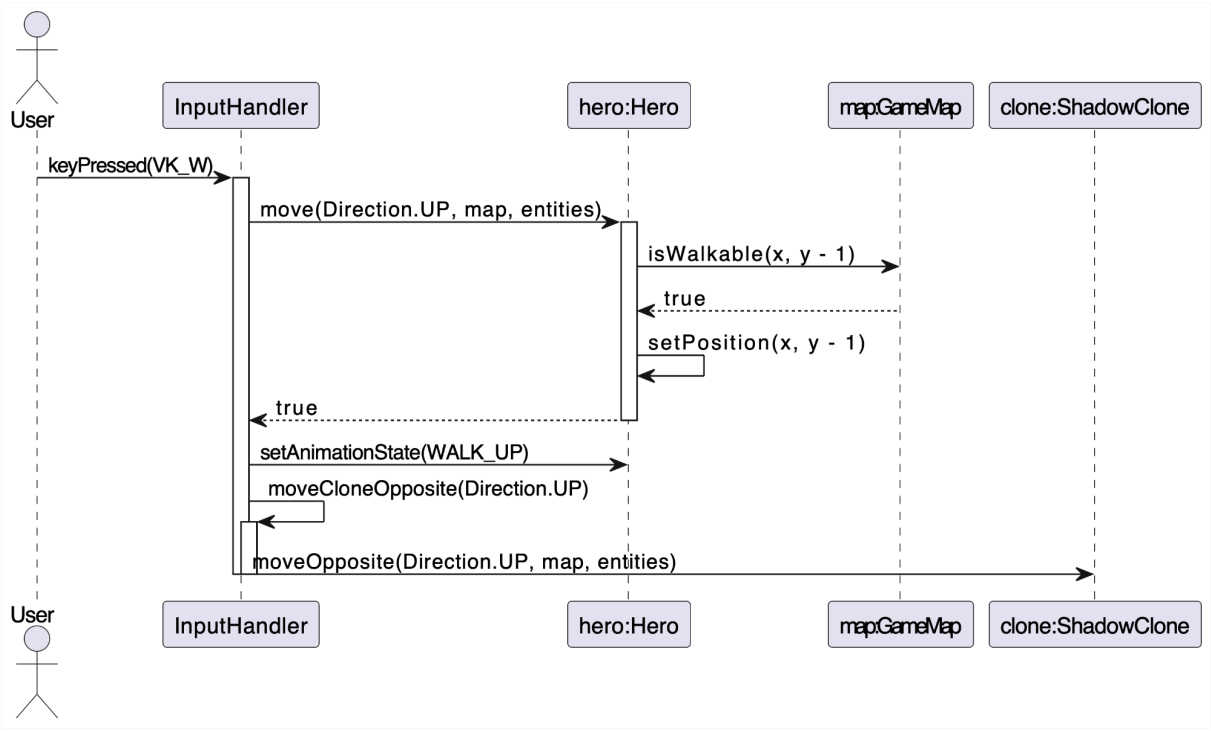
- **Associations:**
    - Permanently associated with distinct GameObject targets to safely apply mathematical state changes and attribute shifts to the underlying simulation model.

Sequence Diagrams:

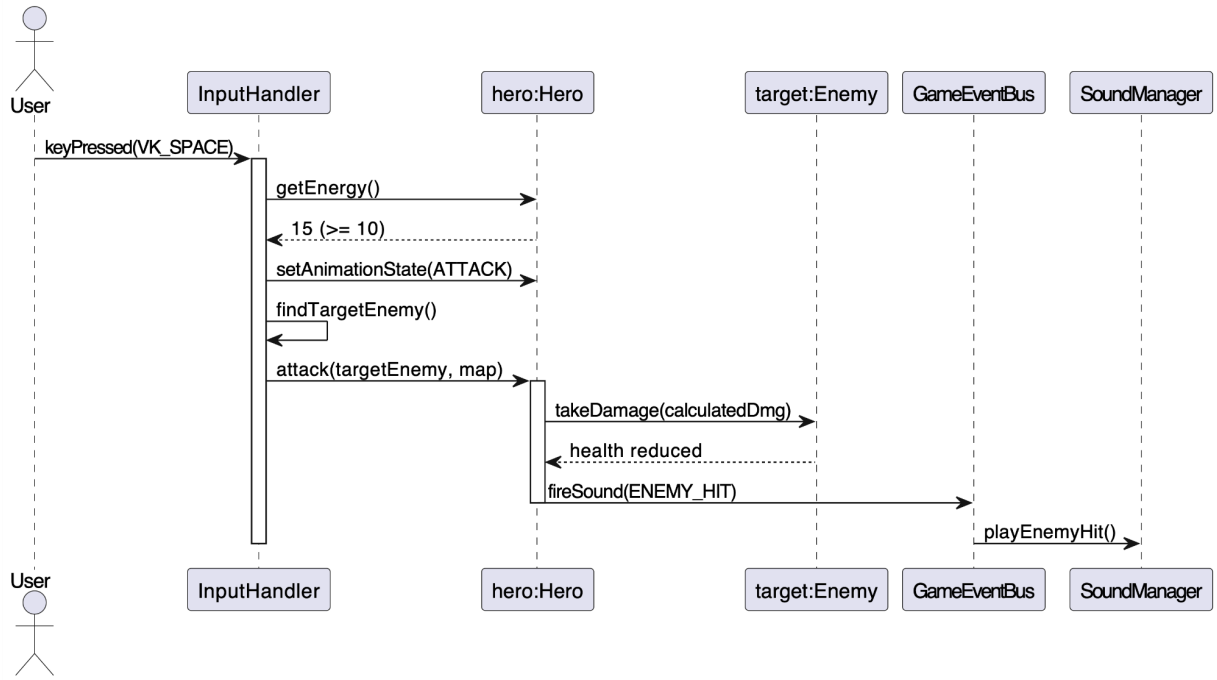
1. Game Initialization Sequence Diagram



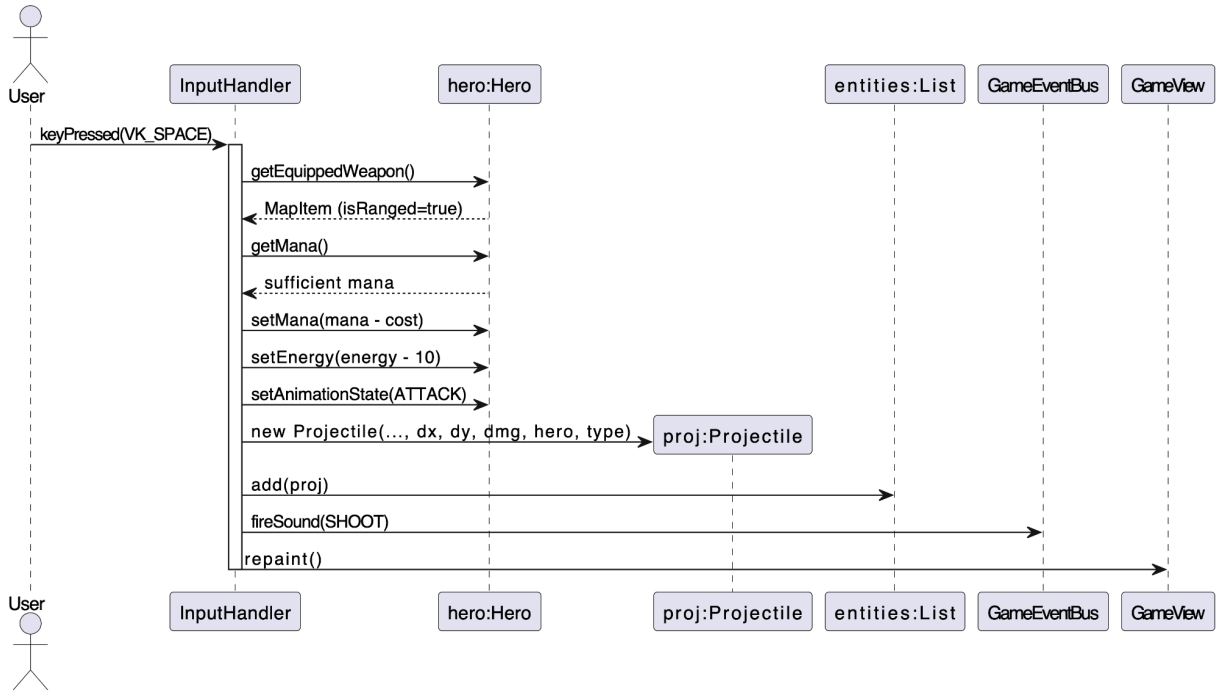
2. Hero Movement Sequence Diagram



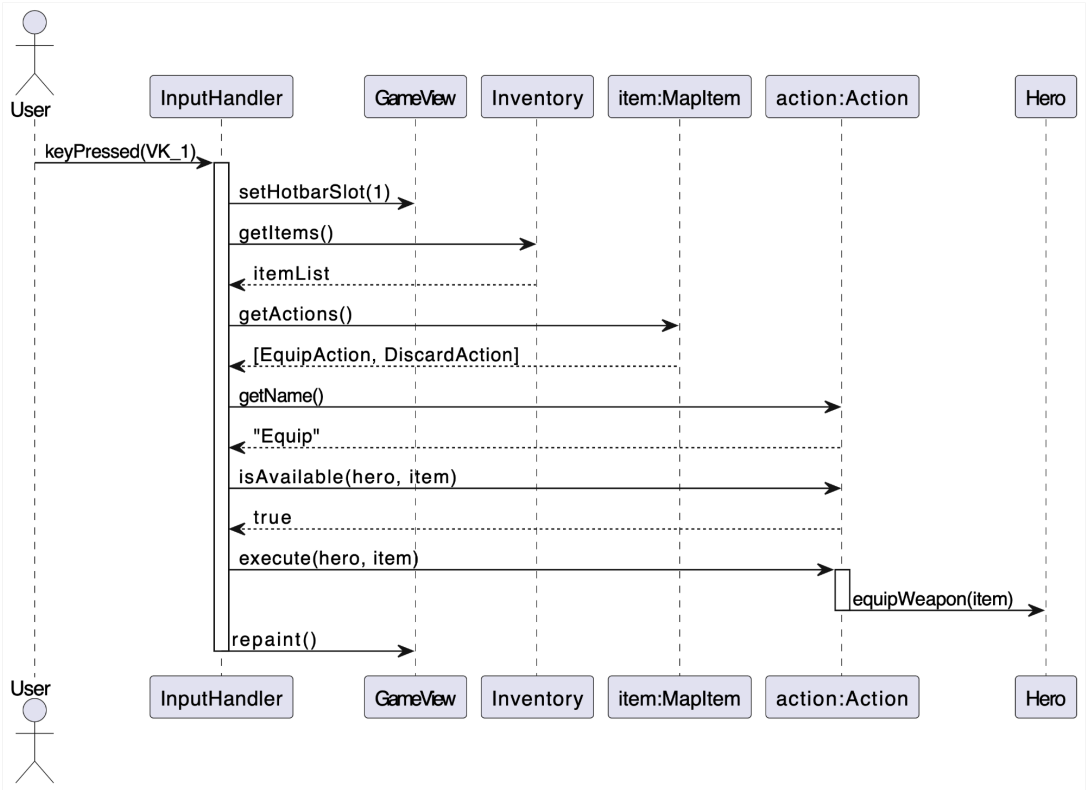
3. Hero Melee Attack Sequence Diagram



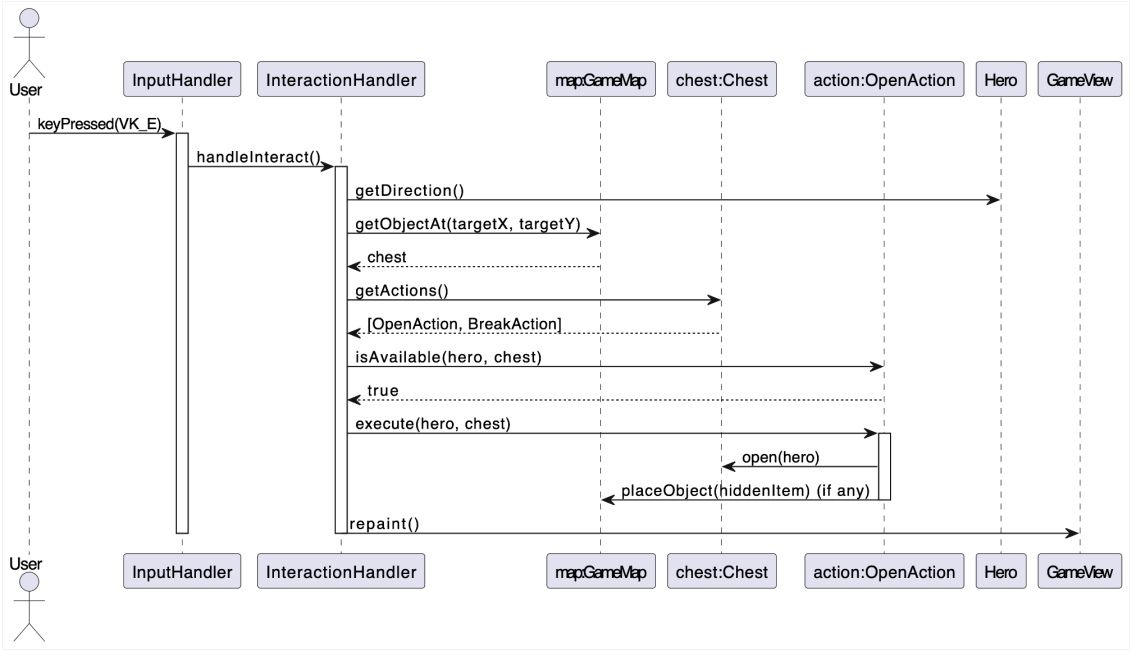
4. Hero Ranged Attack Sequence Diagram



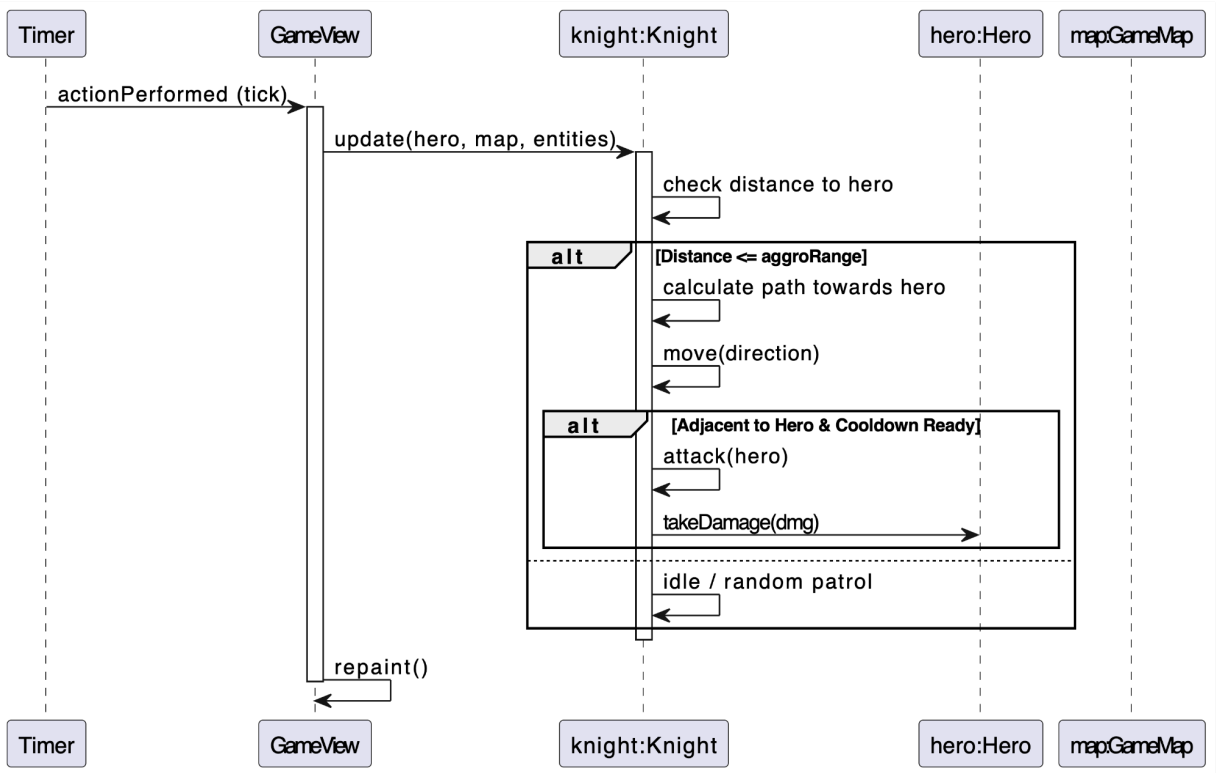
5. Hotbar Item Usage Sequence Diagram



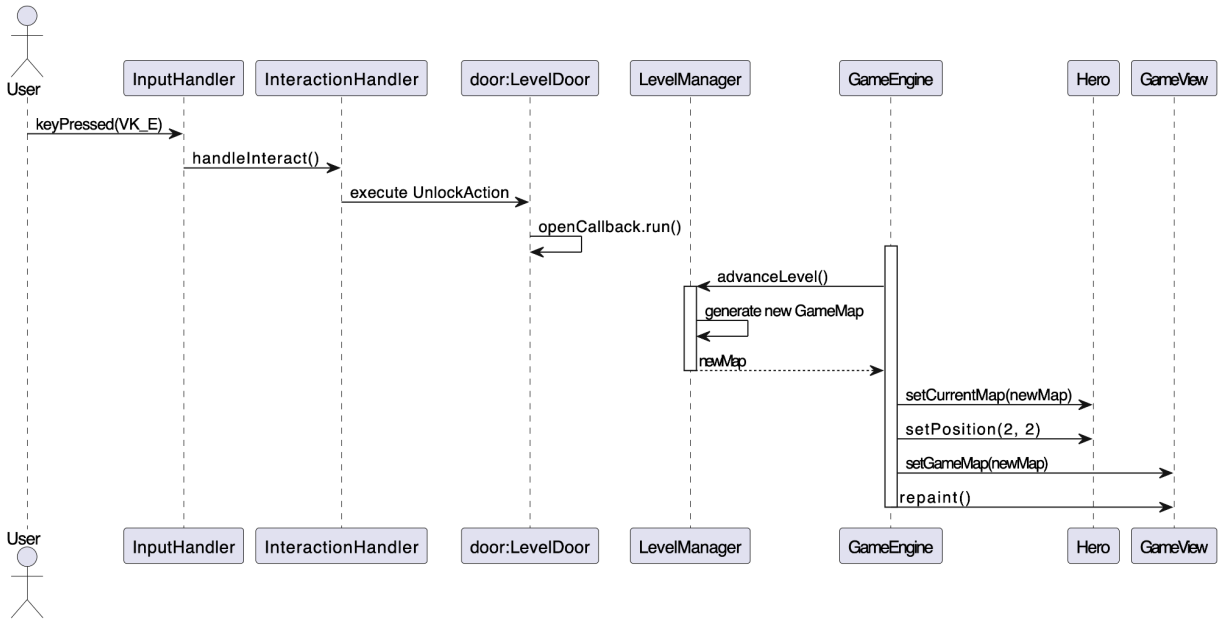
6. Environment Interaction Sequence Diagram



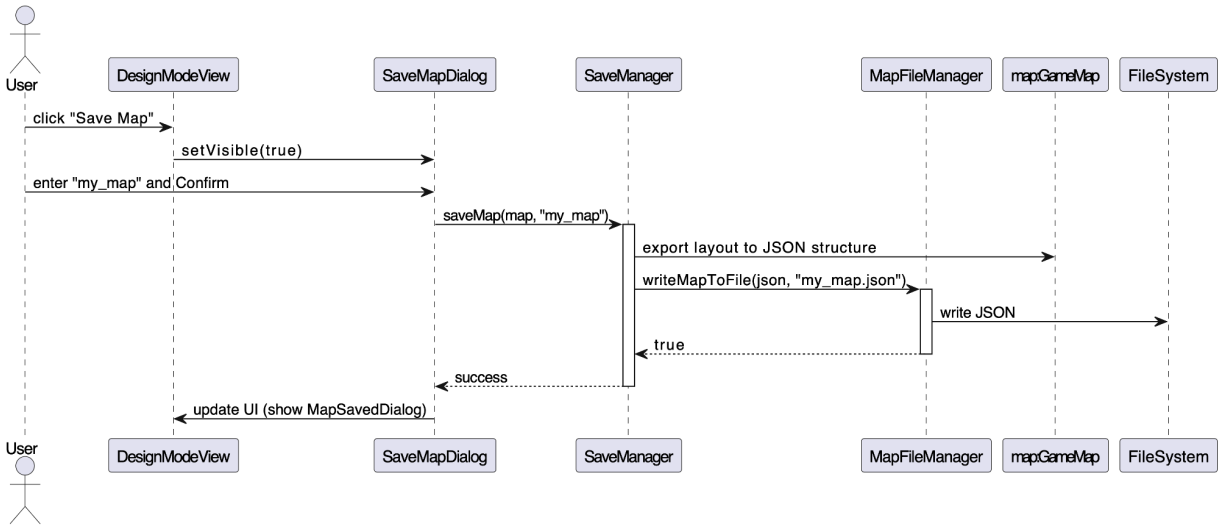
7. Enemy AI Turn Sequence Diagram



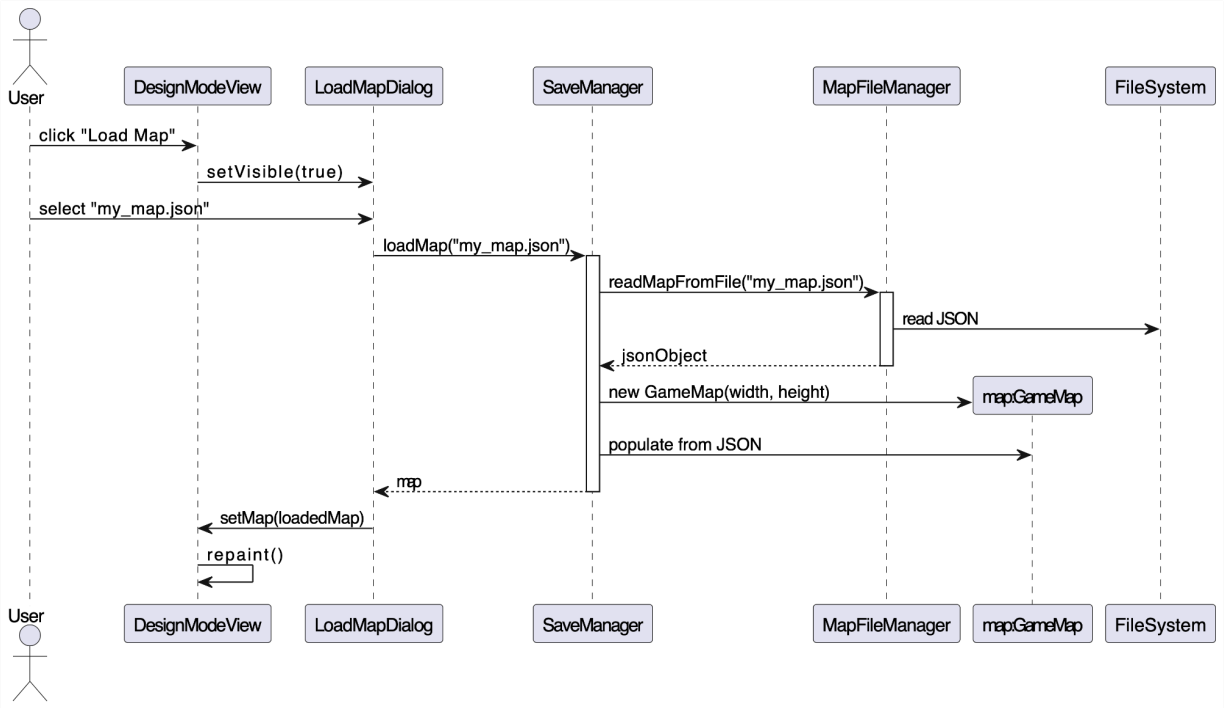
8. Level Transition Sequence Diagram



9. Save Map in Design Mode Sequence Diagram

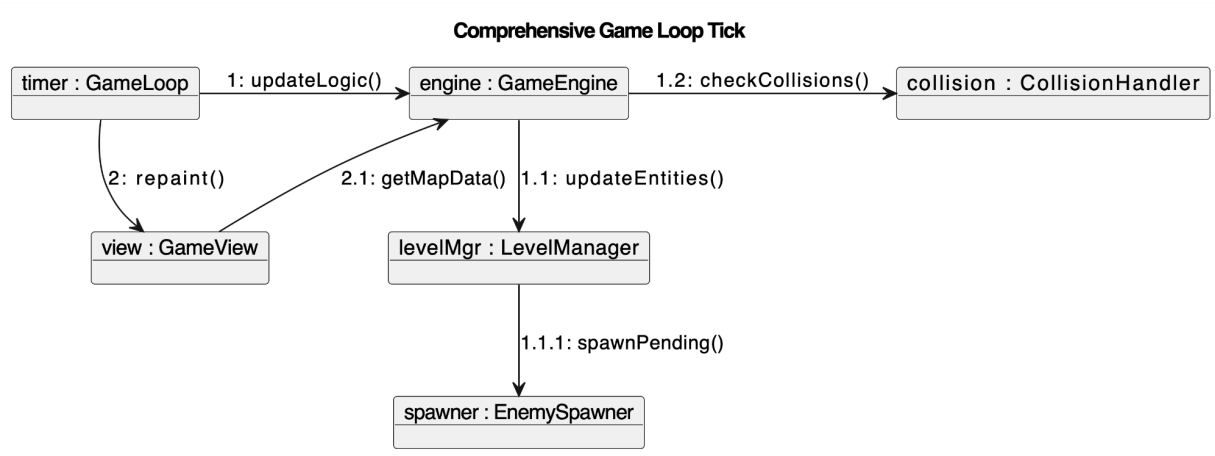


10. Load Map in Design Mode Sequence Diagram

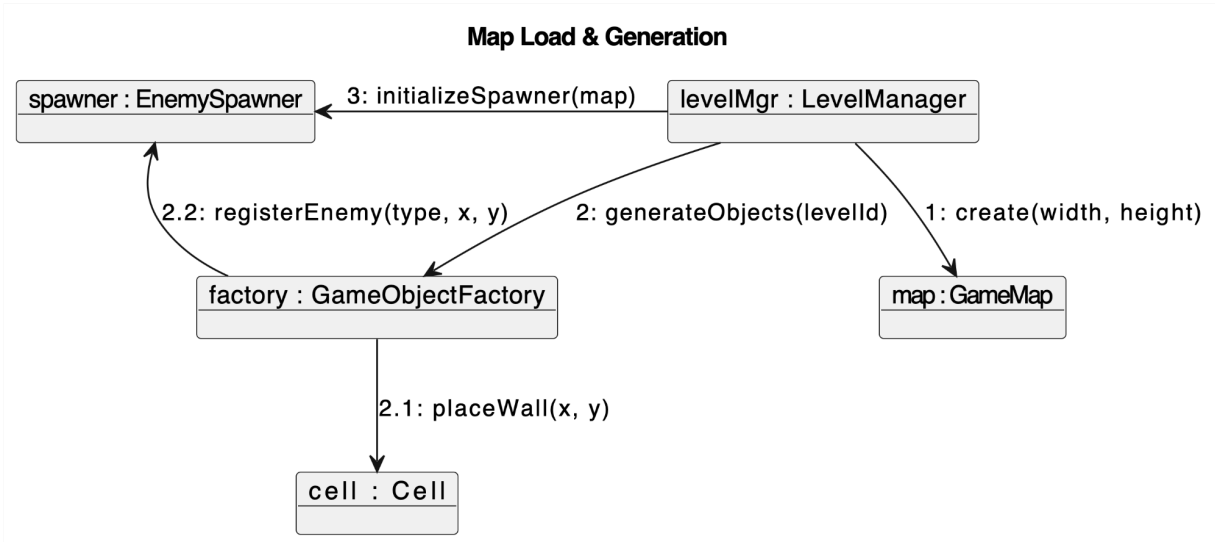


Communication Diagrams:

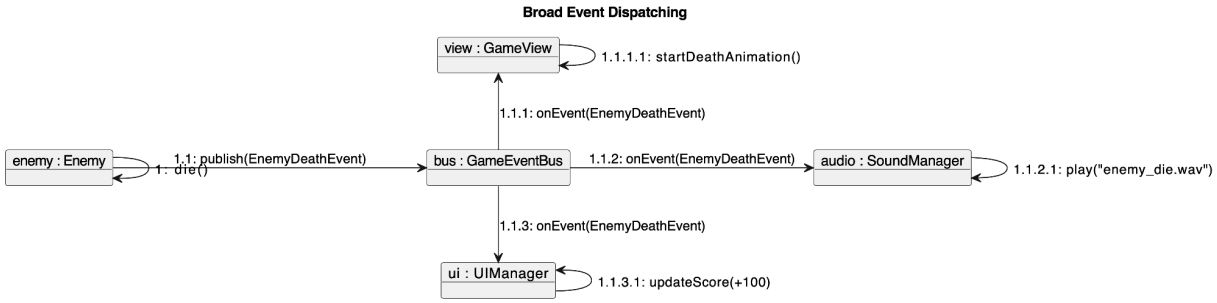
1. Full Tick Update Communication Diagram



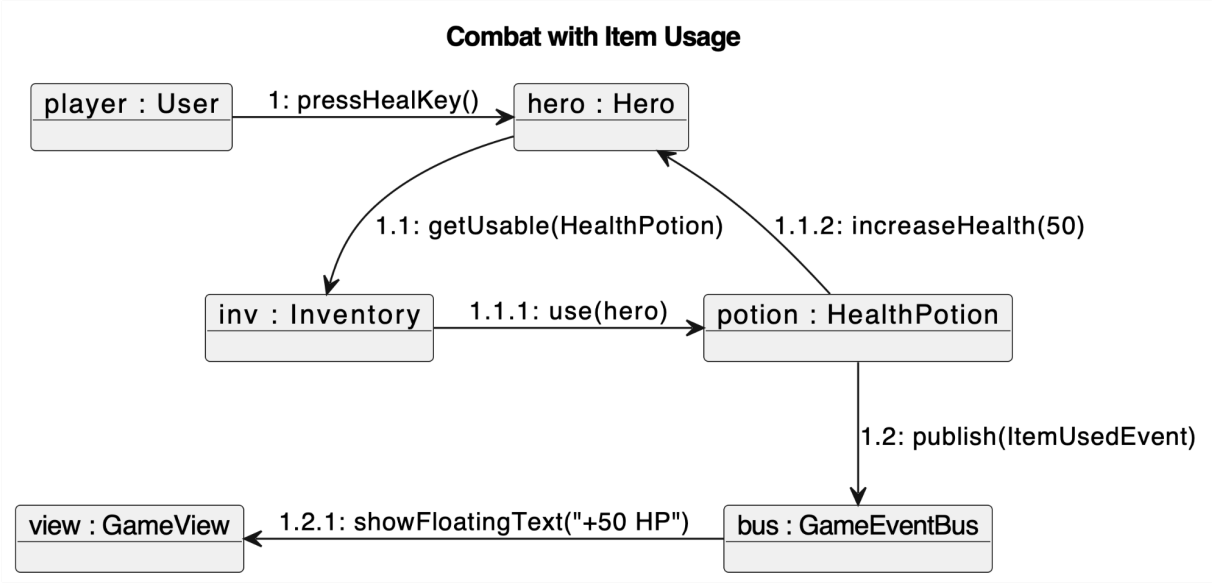
2. Map Factory Generation Communication Diagram



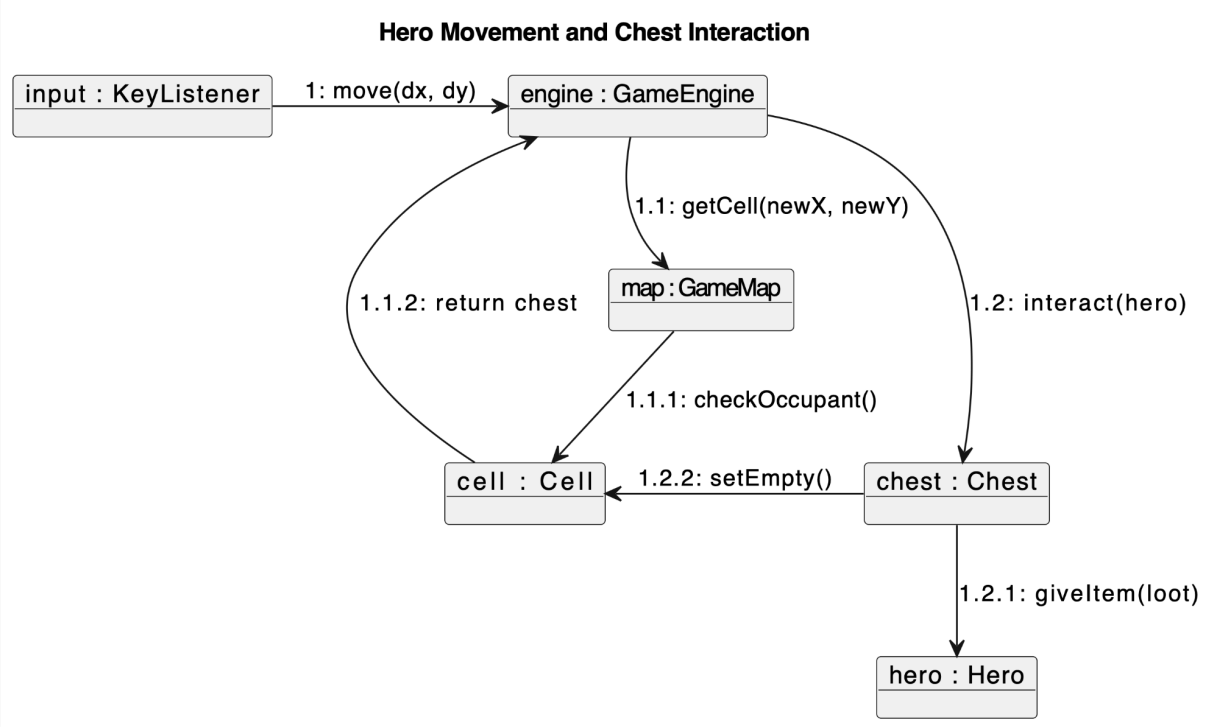
3. Broad Event Dispatching Communication Diagram



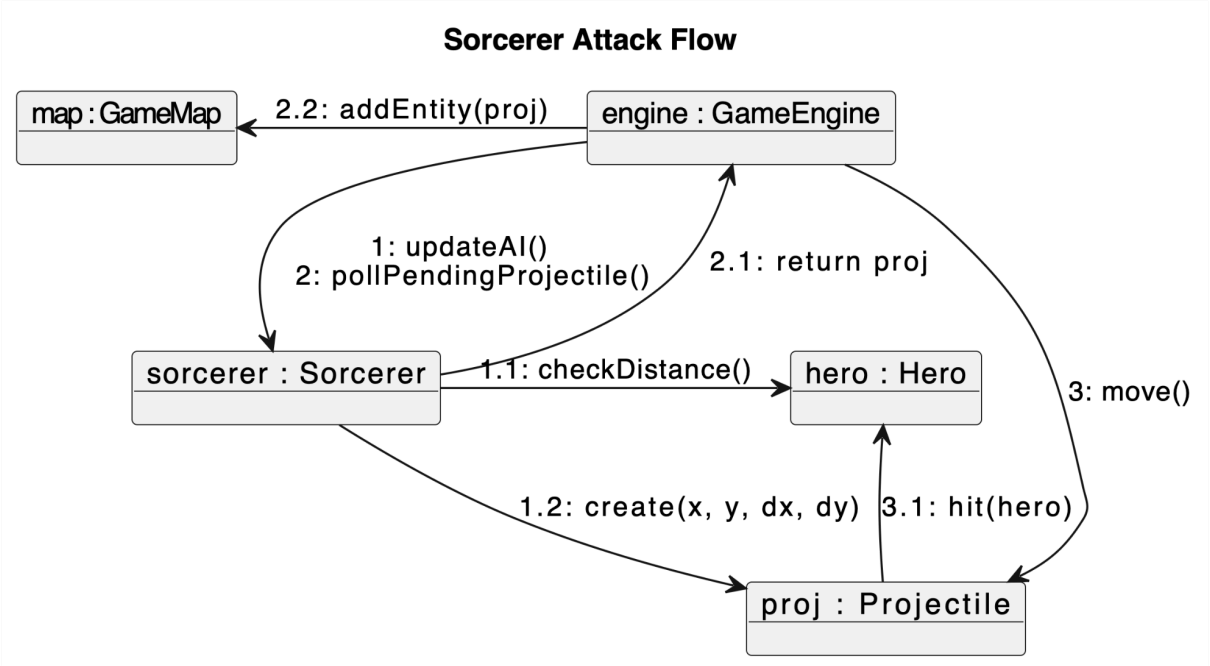
4. Combat & Item Usage Communication Diagram



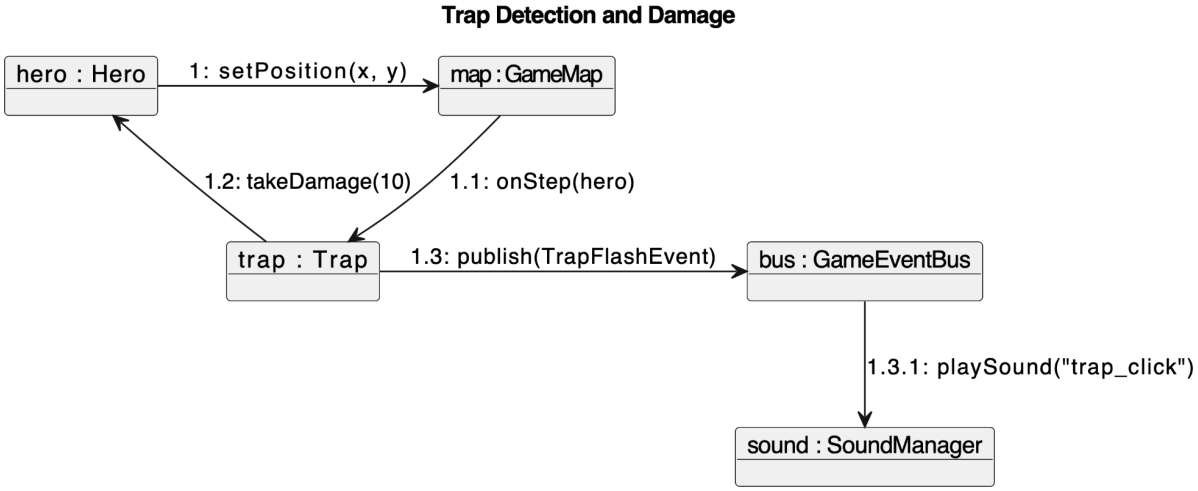
5. Movement & Interaction Communication Diagram



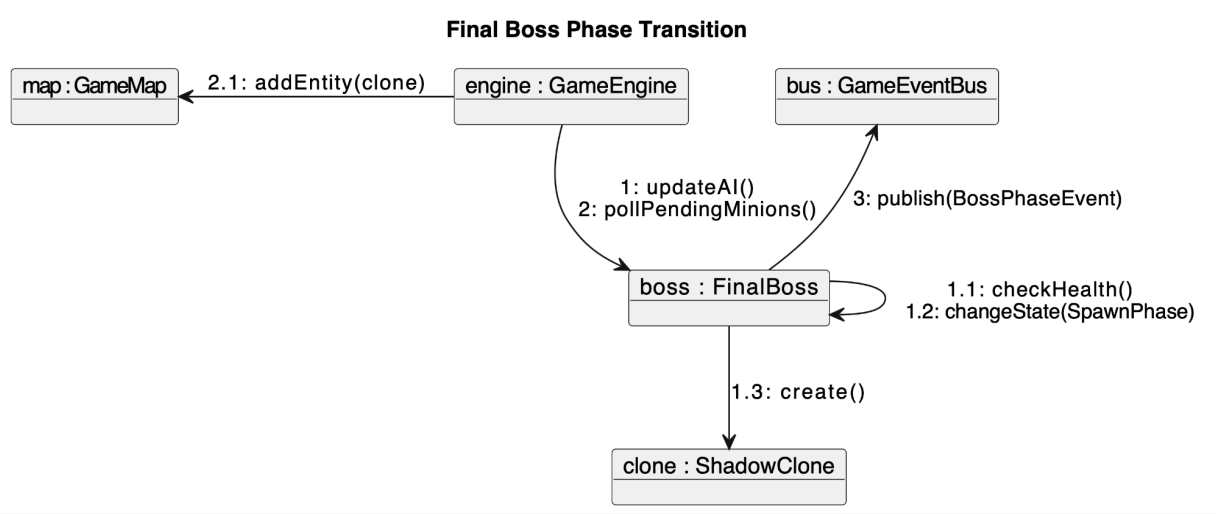
6. Sorcerer Attack Logic Communication Diagram



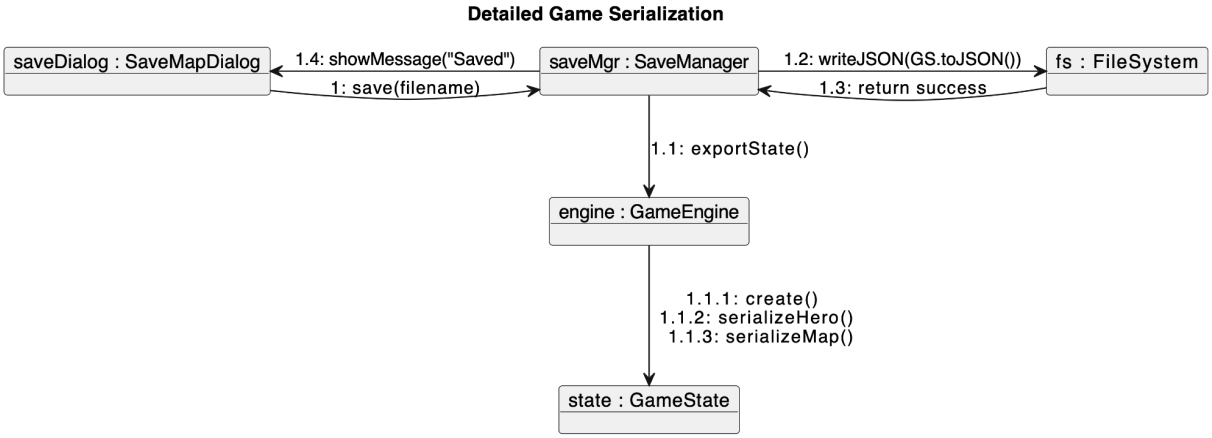
7. Trap Trigger Detection Communication Diagram



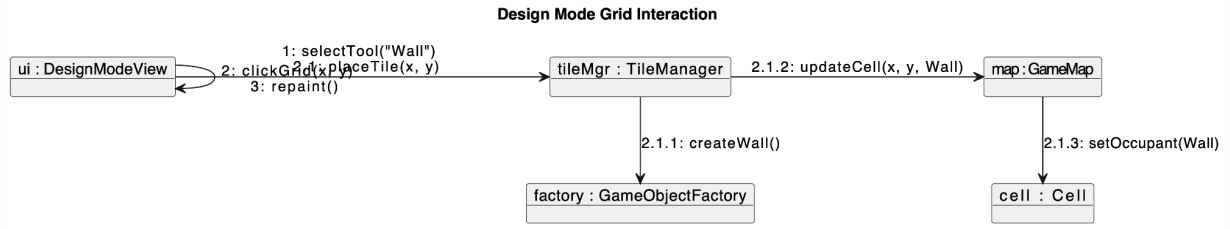
8. Boss Phase Transition Communication Diagram



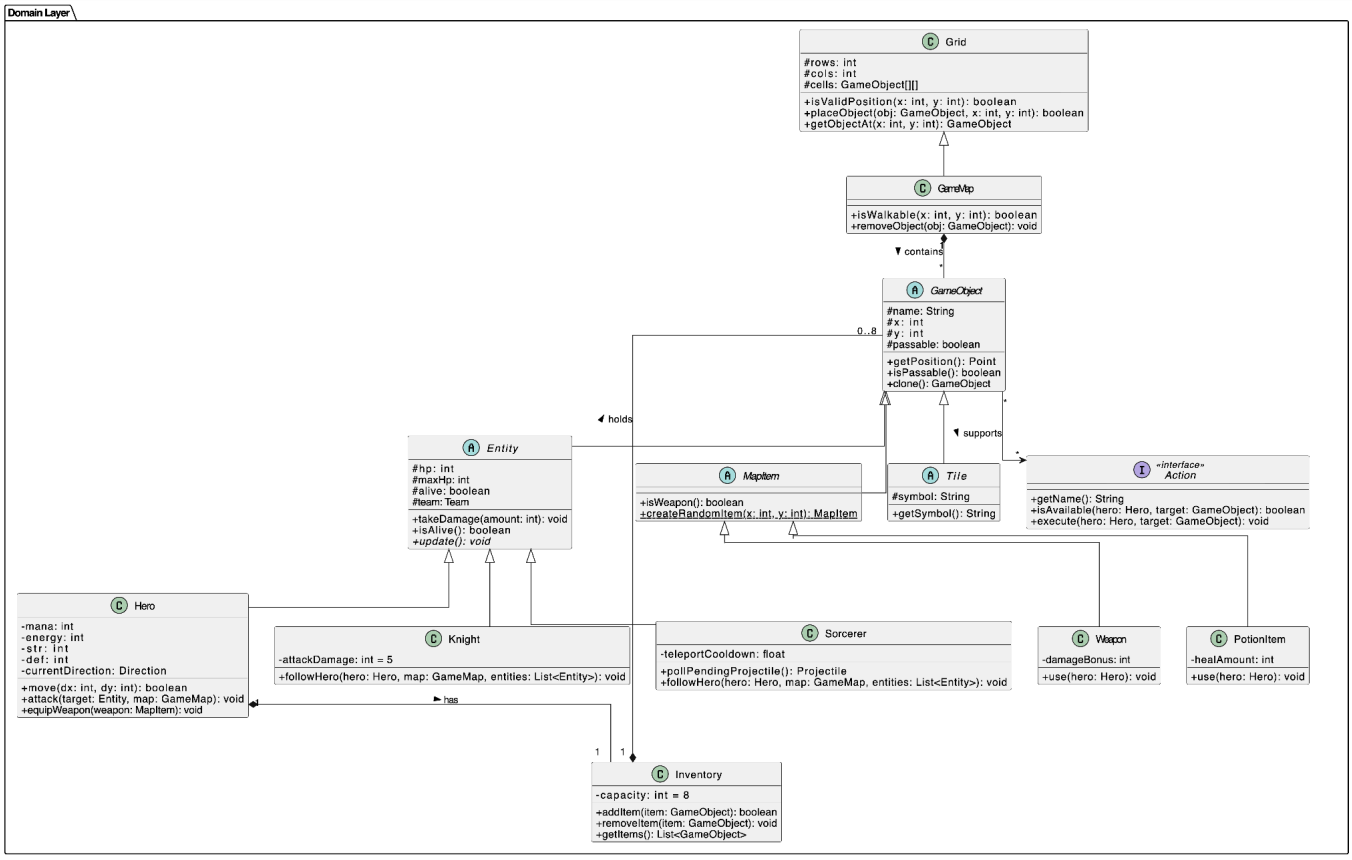
9. Comprehensive Save Serialization Communication Diagram



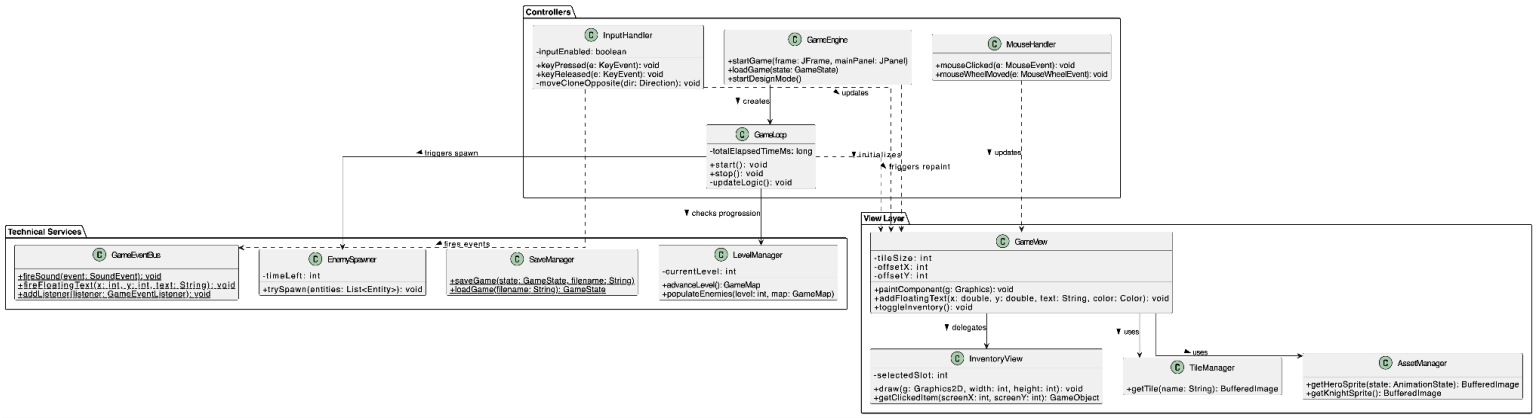
10. Map Editor Grid Interaction Communication Diagram



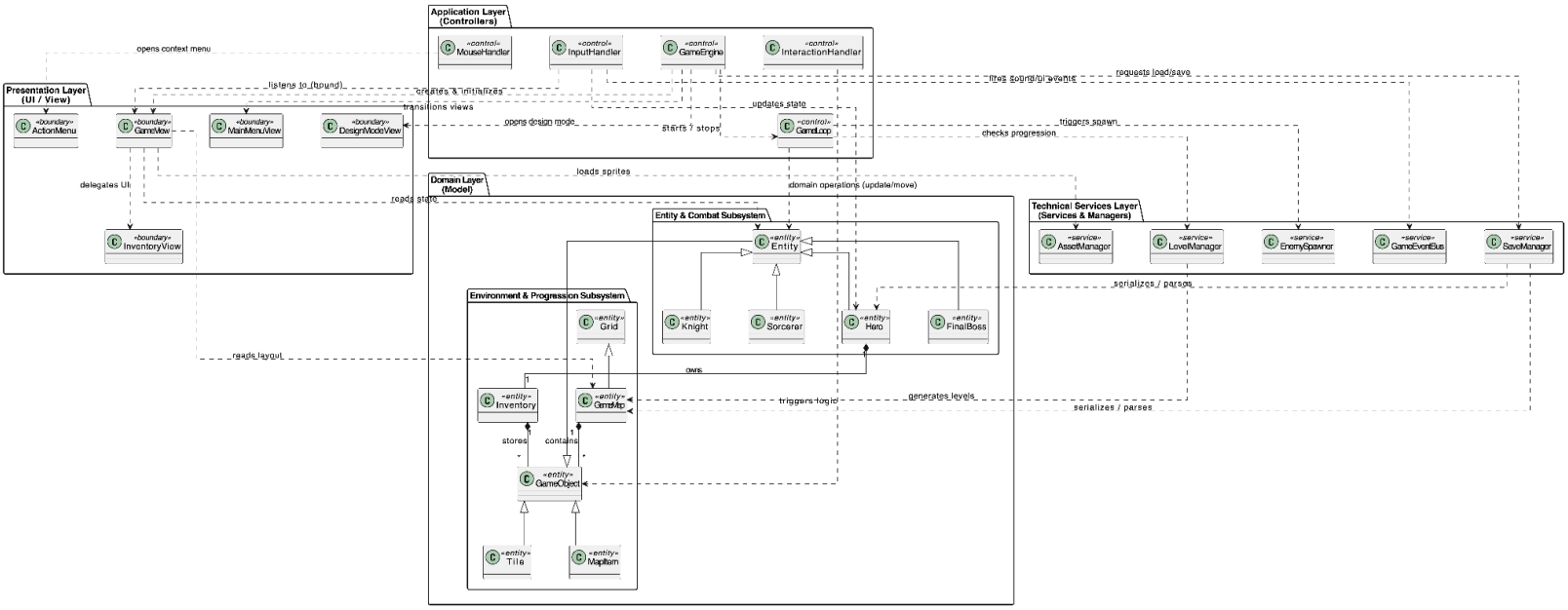
Domain Class Diagram:



Design Class Diagram:



Logical Architecture and Subsystems:



DESIGN ALTERNATIVES: DUNGEON CRAWLER

1. Introduction

During the architectural design phase of the **Dungeon Crawler** project, several alternative approaches were evaluated for core game systems. This document outlines the major design decisions, the alternatives considered, and the rationale behind the final choices. The primary goals driving these decisions were adherence to **GRASP** and **GoF** patterns, high cohesion, low coupling, and the limitations of the Java Swing framework.

2. Core Architecture: Pure OOP (MVC) vs Component-Entity-System (CES)

2.1 The Alternatives

- **Alternative A: Component-Entity-System (CES):** A popular architecture in modern game engines (like Unity) where entities are just IDs, and behaviors/data are strictly separated into Components and Systems.
- **Alternative B: Pure OOP / MVC (Chosen):** Traditional Object-Oriented Programming utilizing deep inheritance trees (e.g., `GameObject` -> `Entity` -> `Hero`) and the Model-View-Controller separation.

2.2 Rationale for Choice

CES offers excellent performance and flexibility for massive games but introduces unnecessary complexity for a grid-based 2D academic project. **Alternative B (MVC)** was chosen because it aligns perfectly with the educational goals of the COMP302 curriculum. It naturally facilitates the demonstration of classic GoF design patterns (like State, Strategy, and Command) and keeps the domain logic cleanly separated from the Swing GUI rendering (`GameView`).

3. Object Interaction: Hardcoded Switch vs Command Pattern

3.1 The Alternatives

- **Alternative A: Switch/If-Else Chains:** Handling interactions by checking `if (obj instanceof Chest)` or `if (obj.getName().equals("Door"))` inside the `InteractionHandler`.
- **Alternative B: Command Pattern (Chosen):** Defining a common `Action` interface with an `execute(Hero, GameObject)` method. Every interactive object provides a list of its available actions.

3.2 Rationale for Choice

Alternative A violates the **Open/Closed Principle**. Every time a new interactive object is added to the game, the central `InteractionHandler` would need to be modified. By choosing **Alternative B (Command Pattern)**, the logic is highly decoupled. To add a new interaction (e.g., "Read" for a scroll), we simply create a new `ReadAction` class. The `InteractionHandler` blindly calls `.execute()`, ensuring the system is open for extension but closed for modification.

4. Audio & UI Feedback: Direct Calls vs EventBus (Observer Pattern)

4.1 The Alternatives

- **Alternative A: Direct Coupling:** The `Hero` or `Projectile` class directly invokes `SoundManager.playSound()` or `GameView.drawFloatingText()` when an event occurs (e.g., taking damage).
- **Alternative B: GameEventBus / Observer (Chosen):** Domain classes fire a generalized event (`new SoundEvent(SoundType.HIT)`) to a central EventBus. The View/Audio managers subscribe to this bus and react accordingly.

4.2 Rationale for Choice

Alternative A creates a disastrously high coupling between the Domain layer and the UI/Audio layers. A `Hero` object should not know about sound drivers or UI panels. **Alternative B** was

chosen to achieve **Low Coupling**. The domain logic simply broadcasts that a "Hit" occurred. The `GameView` listens and draws red floating text, while the `SoundManager` listens and plays a `.wav` file, satisfying Model-View separation.

5. Game Loop: Multithreading vs Swing Timers

5.1 The Alternatives

- **Alternative A: Dedicated Thread Game Loop:** A traditional `while(true)` loop running on a separate `Thread` using `Thread.sleep(16)` to maintain 60 FPS.
- **Alternative B: javax.swing.Timer (Chosen):** Utilizing Java Swing's built-in `Timer` class to trigger logic updates (`actionPerformed`) at fixed intervals.

4.2 Rationale for Choice

While Alternative A is standard for custom game engines, Java Swing is notoriously **not thread-safe**. Updating GUI components or altering lists that the GUI is currently painting from a background thread frequently causes `ConcurrentModificationException`s and visual glitches. **Alternative B** ensures that all game logic and rendering updates occur safely on the Event Dispatch Thread (EDT), guaranteeing application stability.

6. Object Creation: Direct Instantiation vs Factory & Prototype Patterns

6.1 The Alternatives

- **Alternative A: Direct Instantiation:** Using `new HealthPotion()`, `new WallTile()`, etc., scattered throughout map loading and Design Mode logic.
- **Alternative B: Factory & Prototype Patterns (Chosen):** Using a central `GameObjectFactory` to spawn objects based on String keys (e.g., loaded from JSON). For Design Mode, copying objects via the `clone()` method (Prototype).

6.2 Rationale for Choice

The requirement for a Save/Load system (`.json`) and a Custom Map Editor (`.mapjson`) made Alternative A unfeasible. When parsing a JSON string like `"type": "FireWand"`, the system needs a dynamic way to instantiate the object without massive `switch` blocks. The **Factory Pattern** encapsulates creation logic. Furthermore, when a user selects a tile from the Design Mode palette and drags it across the screen, using the **Prototype Pattern**

(`paletteItem.clone()`) is much more efficient and cleaner than calling reflection or complex constructors.

7. Saving Mechanism: Binary Serialization vs JSON

7.1 The Alternatives

- **Alternative A: Java Native Serialization:** Having all classes implement `Serializable` and writing binary `.dat` files using `ObjectOutputStream`.
- **Alternative B: JSON Serialization (Chosen):** Converting the `GameState` into human-readable JSON using a library (e.g., GSON/Jackson) or custom JSON string parsing.

7.2 Rationale for Choice

Native Java serialization is brittle; modifying a class structure often breaks older save files (InvalidClassException). **Alternative B (JSON)** was chosen because it allows players and developers to easily debug, manually edit, or share custom `.mapjson` files. It provides transparency and aligns with modern data interchange standards.

GOF DESIGN PATTERNS ANALYSIS (COMP302 PROJECT)

1. Creational Patterns

1.1. Singleton Pattern

- **Purpose:** To ensure that a class has only one single instance and to provide a global point of access to it.
- **Usage in Project:** `view.AssetManager`
- **Why was it used?** Reloading visual assets (sprites, animation frames, etc.) into memory every time they are needed causes severe performance bottlenecks. By designing the `AssetManager` as a Singleton, all visual assets are loaded into memory exactly once when the game starts. All other classes (like `GameView` or `Entities`) access the same shared instance via `AssetManager.getInstance()` to retrieve sprites.

1.2. Factory Pattern (Simple Factory / Static Factory)

- **Purpose:** To centralize object creation logic in a dedicated class or method, abstracting the instantiation process away from the client code.
- **Usage in Project:** `domain.logic.GameObjectFactory`, `domain.models.item.MapItem`
- **Why was it used?** When loading a saved game, the system needs to instantiate specific Java objects based on string-based data in the save file (e.g., `"DiamondSwordItem"`). The `GameObjectFactory.create(...)` method takes on this exact responsibility (acting as a Pure Fabrication).
- Furthermore, methods like `MapItem.createRandomWeapon()` and `PotionItem.createRandomPotionItem()` act as *Factory Methods*. They encapsulate the complex randomized item-drop logic (based on RNG algorithms) in a single place, promoting High Cohesion and Low Coupling.

2. Behavioral Patterns

2.1. Command Pattern

- **Purpose:** To encapsulate a request or action as an object, making it easier to parameterize, queue, or log requests, as well as support undoable operations.
- **Usage in Project:** The `domain.logic.Action` interface and its concrete subclasses (`TakeAction`, `UseAction`, `BreakAction`, `SearchAction`, `OpenAction`, etc.)
- **Why was it used?** The interactions the player has with items and the environment differ vastly. A sword is "Equipped", a potion is "Used", and a door is "Opened". Instead of writing all these interactions as separate, hard-to-manage methods inside the objects themselves, every distinct action is encapsulated into a *Command* (Action) object.
- **Benefit:** `GameObject`s simply hold a list of their supported actions (`List<Action>`). The user interface doesn't need to know the internal logic of the interaction; it simply retrieves an action and invokes `action.execute(...)`.

2.2. Observer Pattern (Publish-Subscribe)

- **Purpose:** To define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.
- **Usage in Project:** `domain.logic.event.GameEventBus` and `GameEventListener`
- **Why was it used?** This establishes a strict Model-View Separation (a core GRASP principle). For instance, when a hero hits a monster with a sword (Domain Layer), the game needs to render floating red damage text and play a sound effect (View Layer).
- The Domain layer (e.g., `Hero`) should not hold a direct reference to the `GameView`. Instead, the domain publishes an event via `GameEventBus.publish(new FloatingTextEvent(...))`. Since `GameView` implements `GameEventListener`, it listens for these events and executes the drawing logic upon receiving them. This completely decouples the visual rendering from the underlying game logic.

2.3. State Pattern

- **Purpose:** To allow an object to alter its behavior when its internal state changes.
- **Usage in Project:** `domain.models.AnimationState` (IDLE, WALKING, etc.) and `domain.models.GameState`
- **Why was it used?** The sprites rendered for characters (Hero, Knight, Sorcerer) or interactable objects (Chests, Doors) differ depending on what state they are currently in.
- For example, the `getHeroSprite(AnimationState state)` method inside the `AssetManager` decides which animation frame to return based on the character's internal state. The object delegates state-specific logic to the system, which changes its behavior (or rendering output) appropriately.

3. Overall Contribution of GoF Patterns to the Project

Architectural Standard Note: The architectural structures in this project comply with modern software engineering standards and are strongly supported by **GRASP (General Responsibility Assignment Software Patterns)** principles.

1. **Low Coupling:** Through the Observer and Command patterns, direct dependencies between the UI and the Domain have been broken, preventing "spaghetti code".
2. **High Cohesion:** The Factory and Singleton patterns delegate object creation and resource management to highly specialized classes, allowing other classes to focus solely on their primary responsibilities (Single Responsibility).
3. **Extensibility (Open/Closed Principle):** Whenever a new interaction needs to be added to the game, developers only need to create a new class implementing the `Action` interface (e.g., `ReadAction`). The existing code that triggers actions does not require modification.

REFERENCES:

All visual assets (images and sprites) used in the game were obtained from LLM's. and were used in compliance with its license terms for academic, non-commercial use.